Revisión de técnicas de aprendizaje automático y aplicación a la predicción de cancelaciones de reservas de hoteles y la configuración de una política óptima de sobreventa dinámica

Trabajo de Fin de Grado
Curso 2024–2025

Autor
Beñat Perez de Arenaza Eizaguirre

Director
Juan Tinguaro Rodríguez González

Grado en Matemáticas
Facultad de Matemáticas
Universidad Complutense de Madrid

Revisión de técnicas de aprendizaje automático y aplicación a la predicción de cancelaciones de reservas de hoteles y la configuración de una política óptima de sobreventa dinámica

Trabajo de Fin de Grado en Matemáticas

Autor
Beñat Perez de Arenaza Eizaguirre

Director
Juan Tinguaro Rodríguez González

Convocatoria: *Junio 2025*

Grado en Matemáticas
Facultad de Matemáticas
Universidad Complutense de Madrid

29 de junio de 2025

Resumen

Revisión de técnicas de aprendizaje automático y aplicación a la predicción de cancelaciones de reservas de hoteles y la configuración de una política óptima de sobreventa dinámica

Este trabajo estudia estrategias para maximizar el beneficio esperado de un establecimiento hotelero que opera con *overbooking*. Para ello, se da primero un contexto general sobre el aprendizaje automático (*machine learning*) y se presentan algunas de las técnicas más utilizadas dentro del aprendizaje supervisado. Posteriormente, se construye un modelo predictivo con técnicas de aprendizaje supervisado que estima la probabilidad de cancelación de cada reserva. A partir de estas probabilidades, se analizan tres enfoques de gestión: uno conservador, uno con una tasa fija de *overbooking*, y otro dinámico basado en las predicciones del modelo. Se formalizan estrategias que detienen la admisión de reservas cuando la esperanza de ocupación supera la capacidad, o cuando la probabilidad de superarla excede un umbral de riesgo α. Se estudia el impacto del cociente entre beneficio y penalización (B/P), y se comparan los resultados con una estrategia ideal con conocimiento perfecto. Los resultados muestran que el enfoque dinámico, ajustando adecuadamente α, es el más robusto bajo todos los escenarios considerados, y que una tasa de *overbooking* próxima al 50 % es generalmente eficiente.

Palabras clave

Overbooking, optimización del beneficio, modelos de clasificación, estrategias de reserva, gestión del riesgo, aprendizaje supervisado, aprendizaje automático.

Abstract

Review of machine learning techniques and application to the prediction of hotel booking cancellations and the design of an optimal dynamic overbooking policy

This work explores strategies to maximize the expected profit of a hotel establishment operating with overbooking. First, a general overview of machine learning is provided, with a focus on commonly used supervised learning techniques. A predictive model is then built using supervised learning methods to estimate the cancellation probability of each reservation. Based on these probabilities, three management approaches are analyzed: a conservative strategy, a fixed overbooking rate, and a dynamic approach based on model predictions. Strategies are formalized to stop accepting reservations either when the expected occupancy exceeds capacity or when the probability of exceeding it surpasses a risk threshold α. The impact of the benefit-to-penalty ratio (B/P) is studied, and results are compared with an ideal strategy using perfect knowledge. Findings indicate that the dynamic strategy, with properly tuned α, performs best across all considered scenarios, and that an overbooking rate close to 50 % is generally efficient.

Keywords

Overbooking, profit optimization, classification models, booking strategies, risk management, supervised learning, machine learning.

Índice

1. Introducción **1**

2. Preliminares **3**
2.1. Contexto práctico . . . 3
2.1.1. Descripción del problema . . . 3
2.1.2. Referencias relevantes . . . 4
2.1.3. Descripción de los datos . . . 4
2.2. Nociones básicas del aprendizaje automático . . . 5
2.2.1. Introducción al aprendizaje automático . . . 6
2.2.2. Aprendizaje supervisado . . . 6
2.2.3. Parámetros e hiperparámetros . . . 7
2.2.4. Tipos de tareas . . . 7
2.2.5. División de los datos . . . 8
2.2.6. Complejidad . . . 8
2.2.7. Validación . . . 9
2.2.8. Normalización y estandarización . . . 10
2.2.9. Métricas . . . 11

3. Técnicas **14**
3.1. K-NN . . . 14
3.1.1. Introducción y fundamentos . . . 14
3.1.2. Hiperparámetros . . . 15
3.1.3. Ventajas e inconvenientes . . . 15
3.2. Árboles de decisión . . . 16
3.2.1. Introducción y fundamentos . . . 16
3.2.2. Hiperparámetros . . . 18
3.2.3. Ventajas e inconvenientes . . . 18
3.3. Random Forest . . . 18
3.3.1. Introducción y fundamentos . . . 19
3.3.2. Hiperparámetros . . . 19
3.3.3. Ventajas e inconvenientes . . . 20
3.4. Regresión logística . . . 20
3.4.1. Introducción y fundamentos . . . 20
3.4.2. Hiperparámetros . . . 22
3.4.3. Ventajas e inconvenientes . . . 22
3.5. Redes Neuronales . . . 23
3.5.1. Introducción y fundamentos . . . 23

3.5.2. Hiperparámetros 24
3.5.3. Ventajas e inconvenientes 25

4. Estudio Práctico **26**
4.1. Configuración 26
4.1.1. Herramientas utilizadas 26
4.1.2. Preprocesamiento de los datos 26
4.1.3. División de los datos 28
4.1.4. Ajuste de hiperparámetros 28
4.1.5. Hiperparámetros 28
4.1.6. Métricas 29
4.2. Resultados 29
4.2.1. Hiperparámetros seleccionados 30
4.2.2. Mejores resultados 30
4.2.3. Discusión 30
4.3. Estudio de *Overbooking* 31
4.3.1. Planteamiento del problema 31
4.3.2. Implementación de la solución 32
4.3.3. Resultados 35
4.3.4. Discusión 38
4.4. Mejoras futuras 38

5. Conclusión **39**

Conclusión **39**

A. Outliers **42**

B. Código **43**

Índice de figuras

2.1. Curva ROC del modelo. 13
2.2. Área bajo la curva ROC (AUC). 13

4.1. Matriz de confusión. 31
4.2. Beneficio medio y tasa de overbooking para cada valor de B/P. 37
4.3. Relación entre el ratio B/P y α. 37

Índice de tablas

2.1. Matriz de confusión 11

4.1. Rangos de hiperparámetros considerados en cada modelo. 29
4.2. F1-score en validación cruzada y mejores hiperparámetros seleccionados. . . 30
4.3. Métricas de evaluación en test para cada modelo ajustado. 30
4.4. Resultados para valores distintos de B/P (con alpha óptimo). 36

Capítulo 1

Introducción

La inteligencia artificial ha recorrido un largo camino en los últimos años y se ha convertido en un punto vital en todos los sectores, que van desde las finanzas hasta la salud, pasando por el transporte, la educación y el turismo. La combinación de aprendizaje automático y big data ha permitido a las organizaciones automatizar y mejorar la toma de decisiones basadas en datos con mayor precisión. La capacidad de la IA para analizar grandes conjuntos de datos y encontrar patrones complejos entre ellos ha revolucionado la forma en que las industrias abordan los retos que se les presentan, aportando soluciones innovadoras que optimizan recursos, mejoran la experiencia del cliente y generan ventajas competitivas.

En el sector hotelero, la IA y, en particular, las técnicas de aprendizaje automático han irrumpido con fuerza, revolucionando aspectos vitales de la gestión y la operativa. Por ejemplo, la predicción de cancelaciones de reservas se ha convertido en un campo de gran relevancia, puesto que permite a los hoteles anticiparse a las oscilaciones en la demanda y optimizar la asignación de habitaciones. Estudios como [12] han reflejado cómo modelos basados en algoritmos de aprendizaje automático pueden predecir con alta precisión las cancelaciones, ayudando a mitigar pérdidas económicas derivadas de cancelaciones de última hora. De forma similar, la predicción de tasas de ocupación se ha situado como una herramienta clave para la planificación estratégica. Las técnicas de aprendizaje automático permiten a los hoteles prever la ocupación con gran exactitud, facilitando la gestión de precios dinámicos y la optimización de recursos humanos y materiales [6].

Además de mejorar la eficiencia operativa de las organizaciones, la IA tiene un impacto directo en la experiencia del cliente. Por ejemplo, la personalización de ofertas basada en el análisis predictivo posibilita a los hoteles adaptar sus servicios a las preferencias individuales de los huéspedes, lo que aumenta la satisfacción y la fidelidad. También se está empleando para abordar problemas como la sobreventa (del inglés *overbooking*), una práctica común en la industria hotelera que busca maximizar la ocupación, pero que requiere un equilibrio cuidadoso para evitar la insatisfacción del cliente y las pérdidas que esto conlleva. Para esto último se emplea la sobreventa dinámica, que consiste en modificar dinámicamente la tasa de *overbooking* en función de las probabilidades de cancelación.

La relevancia de estas tendencias trasciende el ámbito operativo, ya que también responden a un contexto económico y social en el que la competitividad y la sostenibilidad son prioritarias. En un mundo donde los datos se han convertido en un recurso estratégico, la capacidad de los hoteles para aprovechar la IA no solo optimiza sus procesos, sino que también les permite adaptarse a un mercado en constante evolución, donde las expectativas de los clientes son cada vez más altas. En este sentido, algunas cadenas hoteleras internacionales ya implementan sistemas de IA para personalizar experiencias y optimizar la gestión de inventario, lo cual demuestra el potencial transformador de estas tecnologías.

Entre los principales objetivos de este trabajo, se pueden destacar los siguientes: en primer lugar, este trabajo busca dar un contexto general del aprendizaje automático, que sentará las bases del desarrollo del trabajo. Para ello, se revisan algunas de las nociones

principales del aprendizaje automático, y en concreto del aprendizaje supervisado. Unido a esto, otro de los objetivos principales del trabajo será describir algunas técnicas de aprendizaje automático, como son los árboles de decisión, el *Random Forest*, la regresión logística, el perceptrón multicapa y los K vecinos más cercanos. Finalmente, el trabajo buscará aplicar estas técnicas en la creación de un modelo de predicción de cancelaciones de reservas de la industria hotelera. Además, este modelo se empleará para llevar a cabo un estudio comparativo de estrategias para maximizar los beneficios ajustando la tasa de *overbooking*.

Para ello, la estructura que seguirá el trabajo será la siguiente: en el capítulo 2 se detallarán las nociones básicas del aprendizaje automático, que resultarán fundamentales para los capítulos posteriores. A continuación, en el capítulo 3, se describirán algunas técnicas de aprendizaje supervisado. En el capítulo siguiente, es decir, en el capítulo 4, se expondrá el proceso seguido para la creación de un predictor de cancelaciones de reservas, y se empleará para el estudio de la tasa de *overbooking* bajo diferentes escenarios. Finalmente, en el capítulo 5 se mostrarán las principales conclusiones del trabajo. Para concluir con el trabajo, al final del mismo se incluye un anexo con la definición que se ha tomado de *outliers* (ver apéndice A) y otro donde aparecen las piezas más fundamentales del código que se ha creado para realizar el estudio práctico (ver apéndice B).

Capítulo 2

Preliminares

Esta sección tiene como objetivo los dos siguientes puntos: por un lado, mostrar los detalles principales del problema práctico a tratar, así como detallar la información disponible para abordarlo. Por otro lado, se busca desarrollar las nociones básicas del aprendizaje automático (ML, del término inglés *machine learning*) que sirven de base para facilitar la comprensión del trabajo que se desarrolla en las siguientes secciones.

2.1. Contexto práctico

En esta sección se describirá el problema práctico a tratar, así como su relevancia. Además, se presentarán los estudios relevantes que se han desarrollado en torno al caso práctico. Finalmente, se hará un análisis del conjunto de datos que se empleará para abordar el problema.

2.1.1. Descripción del problema

Los desarrollos tecnológicos de las últimas décadas han cambiado por completo los métodos de reservas de hoteles, haciendo que las conductas de los clientes cambien al mismo tiempo. Hoy en día es habitual el concepto de cancelación gratuita que ofrecen los establecimientos turísticos. Esto resulta beneficioso para el cliente, ya que en caso de surgir un imprevisto, se le devuelve el dinero y no sufre las consecuencias de dicho imprevisto. No obstante, el establecimiento hotelero pierde un cliente que en ocasiones le resulta difícil de reemplazar debido a la poca antelación de la cancelación.

En este contexto, si dichos establecimientos poseyeran la información de si un cliente fuera a cancelar la reserva, podrían tomar las acciones que vieran oportunas para tratar de reducir al máximo posible las potenciales pérdidas que les podría causar una cancelación.

Una medida habitual a este problema suele ser ofertar más habitaciones por encima de la capacidad, que se conoce como *overbooking*. De esta forma, asumiendo que un porcentaje de las reservas va a ser cancelado, los establecimientos pueden intentar paliar los posibles perjuicios de dichas cancelaciones. La clave está en equilibrar la proporción de reservas adicionales ofertadas para que el coste potencial de compensación de una reserva que se esperaba que se cancelara no supere el beneficio obtenido por no dejar habitaciones vacías.

Así pues, uno de los objetivos principales del trabajo será desarrollar un modelo de aprendizaje supervisado que permita predecir, de la forma más fiable posible, si una reserva tiene muchas probabilidades de terminar en cancelación. Además, otro de los objetivos principales del trabajo será ilustrar cómo se aplica este clasificador a la toma de decisiones sobre la tasa de *overbooking* y el impacto que tiene sobre el problema. Para ello, se aplicará el *overbooking* dinámico, que consiste en ajustar de forma continua y adaptativa el número de reservas aceptadas por encima de la capacidad real del hotel, en función de la

probabilidad estimada de cancelación de las mismas.

De esta forma, el desarrollo del trabajo podría repercutir en que los clientes se vieran beneficiados en tanto la oferta de habitaciones aumentaría, lo cual podría implicar una caída de los precios, o una capacidad mayor de elección. Los dueños de los establecimientos, por otro lado, podrán reducir las pérdidas potenciales causadas por cancelaciones de última hora. En suma, ambas partes se podrían ver beneficiadas tras la aplicación del predictor en el problema del *overbooking*.

2.1.2. Referencias relevantes

En el apartado presente se expondrán estudios relevantes que han abordado la aplicación de técnicas de aprendizaje automático en la predicción de cancelación de reservas de hoteles. Estos trabajos sientan las bases del estado actual de este campo y resultan claves para identificar los enfoques y las metodologías aplicadas para abordar el problema.

Nuno Antonio, Ana de Almeida y Luis Nunes [2] muestran la aplicación real en dos hoteles de modelos de aprendizaje automático para predecir las cancelaciones de las reservas. En este proceso llevan a cabo dos contribuciones importantes. Por un lado, se desarrolla una forma de entrenamiento y de ajuste específico para capturar las conductas de cancelación a lo largo del tiempo, así como para aprender de las conductas pasadas y aplicarlas en el futuro. Por otro lado, cabe destacar la creación de una nueva medida llamada *frecuencia mínima* para medir la precisión de las predicciones a lo largo del tiempo.

Sánchez-Medina y C-Sánchez[13] desarrollan un estudio sobre el rendimiento de diversas técnicas de aprendizaje supervisado para la tarea de predecir si una reserva va a ser cancelada o no. Para ello, a diferencia de otros estudios, emplean únicamente 13 variables de entrada, que además coinciden con aquellos datos que se tienden a solicitar a la hora de hacer una reserva de estas características.

Bhardwaj, Yadav y Chaudhary [4] emplean diversas técnicas de aprendizaje automático para tratar de predecir las cancelaciones de hoteles. Entre las técnicas empleadas se pueden encontrar las siguientes: regresión logística, árboles de decisión, bosques aleatorios, clasificador de descenso del gradiente, máquina de soporte vectorial y redes neuronales. Tras la evaluación de dichos modelos, se concluye que las redes neuronales tienen el mejor desempeño, seguido de los árboles de decisión y los bosques aleatorios.

2.1.3. Descripción de los datos

En esta subsección se describirán los datos que se van a emplear para realizar el estudio práctico.

2.1.3.1. Origen de los datos

Los datos que se emplean en este trabajo provienen de un conjunto de datos publicado por Ahsan Raza [11]. Se trata de una recopilación de información sobre reservas de hoteles y se encuentra disponible en *Kaggle*.

2.1.3.2. Variables del dataset

Cada dato está compuesto por 19 variables:

- **Booking_ID**: identificador único de cada reserva.
- **no_of_adults**: número de adultos.
- **no_of_children**: número de niños.
- **no_of_weekend_nights**: número de noches de fin de semana (sabado y domingo) incluidas en la reserva.

- **no_of_week_nights**: número de noches de entre semana (de lunes a viernes) incluidas en la reserva.
- **type_of_meal_plan**: tipo de plan de comida incluido en la reserva.
- **required_car_parking_space**: si la reserva necesita un espacio de aparcamiento.
- **room_type_reserved**: tipo de habitación de la reserva.
- **lead_time**: número de días desde la reserva hasta el día de llegada.
- **arrival_year**: año del día de llegada.
- **arrival_month**: mes del día de llegada.
- **arrival_date**: día del mes de la llegada.
- **market_segment_type**: tipo de reserva (online, offline...).
- **repeated_guest**: si el cliente ya ha tenido una estancia en el mismo sitio alguna vez.
- **no_of_previous_cancellations**: número de reservas canceladas de la persona que reserva.
- **no_of_previous_bookings_not_canceled**: número de reservas no canceladas por la persona que reserva.
- **avg_price_per_room**: precio medio por habitación.
- **no_of_special_requests**: número de peticiones especiales de la persona que reserva (habitación con buenas vistas,...).
- **booking_status**: indica si la reserva fue cancelada o no.

2.1.3.3. Número de observaciones

El conjunto de datos contiene un total de 36.275 observaciones, cada una de las cuales tiene un identificador único.

2.1.3.4. Características generales del dataset

Existen dos clases: el 1 indica la ausencia de cancelación y el 0 la presencia en la variable *booking_status*. Se puede observar que las clases están desbalanceadas, puesto que en torno a un tercio de las reservas ha sido cancelada, mientras que el resto no. Cabe destacar que no faltan valores en el conjunto de datos. Existen 4 atributos categóricos y el resto son numéricos.

Asimismo, no existen variables con una alta correlación entre ellas, lo cual indica que no se pueden descartar atributos sin hacer un análisis más exhaustivo de los mismos. Por otro lado, se puede apreciar que existen ciertas variables que no mantienen correlación alguna con la variable a predecir. Aunque estas variables podrían no estar directamente correlacionadas con dicha variable, podría existir una correlación indirecta mediante otros predictores, por lo que *a priori* tampoco se podrían obviar sin que esto perjudicara el rendimiento final del predictor.

2.2. Nociones básicas del aprendizaje automático

En esta subsección se abordarán los conceptos fundamentales del aprendizaje automático, que sentarán las bases técnicas y formales de las secciones posteriores.

2.2.1. Introducción al aprendizaje automático

El ML es un campo de la inteligencia artificial (IA, del término inglés *artificial intelligence*) enfocado en el estudio y desarrollo de métodos computacionales que permiten mejorar el rendimiento de un ordenador para desempeñar una tarea práctica usando datos, información y/o experiencias. Para ilustrarlo en un ejemplo cotidiano, como se muestra en [9], si se tuviera que describir la cara de una persona para que otros pudieran reconocerla, resultaría complejo hacerlo con palabras. No obstante, disponer de fotografías de su cara facilitaría este proceso, lo cual es precisamente lo que se intenta emular en ML. Existen tareas altamente complejas de describir, en parte, debido al alto número de variables de las que dependen. En este sentido, resulta más conveniente hacer uso de ejemplos de resolución de tales tareas para captar su esencia. Así, el ordenador dispondría del conocimiento necesario para saber si un nuevo ejemplo tiene la misma esencia que los anteriores.

2.2.1.1. Paradigmas del aprendizaje automático

Dentro del ML existen tres paradigmas principales: el aprendizaje supervisado (SL, del término inglés *supervised learning*), el aprendizaje no supervisado (UL, del término inglés *unsupervised learning*) y el aprendizaje por refuerzo (RL, del término inglés *reinforcement learning*). Cada uno de ellos se adapta a distintos tipos de datos y objetivos, por lo que, dependiendo de la naturaleza del problema a solucionar, se recurrirá a uno de los enfoques. A continuación, se hará una breve descripción de cada uno de ellos.

Si se quiere predecir una variable de salida basado en ciertas variables de entrada (o atributos) y se dispone de un conjunto de ejemplos cuya variable a predecir es conocida, es decir, un conjunto de ejemplos etiquetados, se trata de un problema de SL, tal y como se muestra en [15]. El objetivo será crear un procedimiento con base en este conjunto de elementos etiquetados que servirá para predecir el valor de la variable de salida de nuevos ejemplos, de los cuales sólo se conocen los valores de los atributos.

En el caso de que se pretenda predecir una variable de salida basado en ciertos atributos pero los ejemplos de los que se dispone no están etiquetados, entonces se trata de UL, como se describe en [15]. En este caso, el objetivo no es etiquetar los nuevos ejemplos, sino encontrar patrones o estructuras útiles de cara a, entre otras cosas, simplificar, depurar o representar alternativamente los elementos.

Por último, como se observa en [7], el RL hace referencia al conjunto de problemas de ML en los que se tiene un agente autónomo, que explora su entorno, recibe información sobre su estado actual y toma decisiones en base a ello. El entorno le otorga una serie de recompensas (positivas o negativas) en función de sus acciones y el objetivo del agente es maximizar las recompensas obtenidas. El RL muestra que no todas las técnicas de ML emplean datos para aprender.

2.2.2. Aprendizaje supervisado

El presente trabajo se centrará en el primero de estos paradigmas, es decir, en el SL. Como ya se ha presentado, bajo este paradigma se trabaja con datos etiquetados. El conjunto de datos, que se denotará por $\mathcal{D}$, está formado por ejemplos u observaciones caracterizados por pares de atributos X_i y salidas Y_i

$$\mathcal{D} = \{(X_i, Y_i)\}_{i=1}^{n},$$

donde:

- $X_i \in R^p$ es el vector de atributos de la i-ésima instancia, donde p representa el número de atributos que describen cada instancia.
- $Y_i \in R$ es el valor de la variable de salida correspondiente.
- n es el tamaño del conjunto de datos.

Se asume que existe un patrón desconocido f que relaciona los atributos de los ejemplos con la variable de salida $Y = f(X) + ruido$, donde el ruido es la variabilidad en los datos que no puede ser explicada por el patrón. El objetivo principal del SL será encontrar un procedimiento $\hat{f}$ que aproxime f, de tal forma que $\hat{Y} = \hat{f}(X)$, donde $\hat{Y}$ será la variable de salida predicha.

En el caso de que el procedimiento se trate de un modelo, se tratará de ajustar los parámetros θ de dicho modelo $\hat{f}_\theta(X)$. Para ello, se empleará una función de pérdida $L(Y, \hat{f}_\theta(X))$ y se buscarán los parámetros θ que la minimicen.

$$\hat{\theta} = \arg\min_{\theta} \sum_{i=1}^{n} L(Y_i, \hat{f}_\theta(X_i))$$

Un algoritmo de aprendizaje se caracteriza por la forma del modelo y su optimización, de modo que diferentes modelos y técnicas para su optimización resultan en diferentes técnicas y/o algoritmos de aprendizaje.

2.2.3. Parámetros e hiperparámetros

Relacionado con esto, cabe destacar la diferencia entre los parámetros e hiperparámetros. Los primeros son valores internos del modelo que se ajustan durante el entrenamiento para mejorar las predicciones. En cambio, los segundos son valores externos al modelo, que deben ser definidos antes de iniciar el entrenamiento y afectan el proceso de optimización del modelo [17]. Por ejemplo, si se desea entrenar un modelo que recomiende películas, un hiperparámetro podría ser el número de características que usará el procedimiento. Un número reducido de características, como solamente usar el título y la fecha de lanzamiento, podría ser insuficiente para predecir correctamente las películas de interés para un usuario. Sin embargo, un número excesivo de características, como el color predominante de la portada, podría llevar a ajustarse demasiado a los datos de entrenamiento, lo cual afectaría negativamente a la capacidad predictiva ante datos nuevos.

2.2.4. Tipos de tareas

Dependiendo de la naturaleza de la variable a predecir, se pueden diferenciar dos tipos de tareas. Si la variable de salida es de tipo cualitativo, se tratará de un problema de clasificación. De lo contrario, si la variable tiene una naturaleza cuantitativa, se tratará de un problema de regresión.

2.2.4.1. Clasificación

La primera es la tarea de clasificación, cuyo objetivo es encontrar un procedimiento $\hat{f}$ que mapea los atributos de los ejemplos a una variable de respuesta que tomará valores en un conjunto discreto $\mathcal{C}$, es decir:

$$Y_i \in \mathcal{C} = \{c_1, c_2, \ldots, c_k\}$$

donde k es el número de elementos posibles. Cada uno de estos valores se conocerá como clase [8]. Por ejemplo, una tarea podría ser predecir si una persona sufrirá o no un ataque al corazón (las dos posibles clases que puede tomar la variable de salida) dados sus valores biométricos (atributos). Dentro de este tipo de tareas, se pueden distinguir aquellas que buscan clasificar el conjunto de datos en dos clases (clasificación binaria) o en más de dos (clasificación multiclase).

Sin embargo, el procedimiento podría ver disminuido su poder de predicción en caso de que el número de ejemplos de las clases no fuera uniforme, lo que se conoce como desbalance de clases. Esto puede llevar al modelo a ignorar la clase minoritaria [1]. Por ejemplo, en un conjunto de datos de pacientes con una enfermedad poco común, el procedimiento podría

centrarse en los casos negativos y pasar por alto los positivos. Para resolver esto, se pueden aplicar técnicas como el sobremuestreo de la clase minoritaria o el submuestreo de la clase mayoritaria, ambas útiles para equilibrar las clases [3].

Cabe destacar que la mayoría de las técnicas utilizadas para este tipo de problema proporcionan un resultado o valor intermedio que puede ser entendido como la probabilidad de la clase positiva (o de cada clase en multiclase). A partir de este resultado intermedio se suele generar la predicción de una clase tomando la clase del máximo de los valores atribuidos a cada clase. En el caso binario, esto se puede llevar a cabo empleando un umbral u de manera que, si la probabilidad predicha de la clase positiva es mayor que u entonces la instancia se predice como positiva.

2.2.4.2. Regresión

El otro tipo de tarea supervisada es la regresión. En este caso, no se pretende categorizar o clasificar cada ejemplo en una clase, sino asignarle un valor continuo concreto $Y_i \in R$. Es el caso de atribuir a cada casa, según sus características (como el tamaño, el número de habitaciones, etc.), un valor de mercado concreto.

2.2.5. División de los datos

El objetivo principal del procedimiento $\hat{f}$ es ser útil para resolver un problema práctico. Es por esto que de nada sirve que dicho procedimiento prediga correctamente la variable de salida de los datos etiquetados, si para un nuevo ejemplo no etiquetado no es capaz de predecir adecuadamente el valor de su variable de salida. Por tanto, resulta de vital importancia que la evaluación del procedimiento se haga con datos que no se hayan utilizado para ajustar el mismo. De lo contrario, podría parecer que el rendimiento predictivo fuera perfecto cuando realmente dista de serlo. Por ello, los datos etiquetados deben dividirse en dos conjuntos: un conjunto de entrenamiento (que servirá para ajustar el procedimiento) y un conjunto de prueba (con el que se evaluará la capacidad predictiva del procedimiento)

$$\mathcal{D}_{\text{ent}} = \{(X_i, Y_i)\}_{i \in \mathcal{I}_{\text{ent}}}, \quad \mathcal{D}_{\text{pru}} = \{(X_i, Y_i)\}_{i \in \mathcal{I}_{\text{pru}}},$$

donde:

- $\mathcal{I}_{\text{ent}}, \mathcal{I}_{\text{pru}} \subseteq \{1, 2, \ldots, n\}$,
- $\mathcal{I}_{\text{ent}} \cap \mathcal{I}_{\text{pru}} = \emptyset$,
- $\mathcal{I}_{\text{ent}} \cup \mathcal{I}_{\text{pru}} = \{1, 2, \ldots, n\}$.

2.2.6. Complejidad

La complejidad de un procedimiento mide la capacidad del mismo para aproximar funciones. Si la complejidad es alta, entonces tendrá mucha flexibilidad para generalizar el ruido y no sólo el patrón, por lo que al enfrentarse a datos nuevos con un ruido diferente, el rendimiento predictivo decae, potencialmente de manera grave. Esto es lo que se conoce como sobreajuste (del término inglés *overfitting*) [5]. Por el contrario, si la complejidad es baja, el procedimiento tendrá poca flexibilidad. Esto no permite capturar el patrón, lo que se conoce como subajuste (del término inglés *underfitting*). La complejidad determina la flexibilidad del procedimiento, y los hiperparámetros son precisamente los que permiten controlarla. Por ejemplo, un polinomio lineal tiene menos flexibilidad que un polinomio cuadrático para ajustar datos complejos, y viceversa.

La excesiva flexibilidad del modelo captura el ruido, es decir, la variabilidad de los datos que no puede ser explicada por el modelo. Para reducir el sobreajuste, se debe limitar la flexibilidad del modelo, penalizando una alta complejidad.

2.2.6.1. Regularización

Una manera de lograr esto es introduciendo un término en la función de pérdida que penalice la complejidad del modelo. Esto es lo que se conoce como regularización

$$\hat{\theta} = \arg\min_{\theta} \left\{ \sum_{i=1}^{n} L\big(Y_i, \hat{f}_\theta(X_i)\big) + \lambda\, \mathcal{R}(\theta) \right\},$$

donde:

- $\mathcal{R}(\theta)$ es el término de regularización que mide la complejidad del modelo en función de los parámetros θ.
- $\lambda \geq 0$ es el hiperparámetro de regularización que controla el peso del término de regularización frente a la pérdida.

Esta idea se alinea con el principio de la navaja de Ockham, que sugiere preferir la explicación más simple cuando varias son posibles [14].

2.2.7. Validación

Esta relación entre hiperparámetros y complejidad plantea la necesidad de un tercer conjunto de datos: el conjunto de validación. Para obtener los hiperparámetros que llevan a que un modelo tenga mayor capacidad de predicción, pasa algo similar a lo que sucedía con la necesidad de separar el conjunto de datos en entrenamiento y prueba. Tras ajustar una familia de modelos con diferentes valores de hiperparámetros, se debe comprobar cuál es la mejor configuración de todas ellas. Para ello, se deben emplear unos datos que no se han utilizado para ajustar los propios modelos, con la intención de poder ver la verdadera capacidad de predicción que tienen. Por tanto, queda claro que los hiperparámetros no se deben ajustar con el conjunto de entrenamiento.

Sin embargo, tampoco se deben ajustar con el conjunto de prueba. En tal caso, sucedería algo similar a lo recién expuesto. Si se optimizara la configuración de hiperparámetros con el conjunto de prueba, el rendimiento de tal modelo sería probado con el mismo conjunto, por lo que podría suceder que se sobreestimara el rendimiento predictivo del modelo. Por ello, es necesario que, adicionalmente al conjunto de entrenamiento y de prueba, se emplee lo que se conoce como conjunto de validación.

En suma, para seleccionar un modelo y su configuración de hiperparámetros dentro de una familia de modelos y configuraciones, una vez entrenados con el conjunto de entrenamiento, se prueba el poder predictivo de cada uno con el conjunto de validación. Entre todas las configuraciones de la familia, se elige aquella que obtiene el mejor rendimiento, el cual se evaluará mediante métricas que se presentarán a continuación. Finalmente, se evaluará el rendimiento de la configuración seleccionada en el conjunto de prueba [10].

$$\mathcal{D}_{\text{ent}} = \{(X_i, Y_i)\}_{i \in \mathcal{I}_{\text{ent}}}, \quad \mathcal{D}_{\text{pru}} = \{(X_i, Y_i)\}_{i \in \mathcal{I}_{\text{pru}}}, \quad \mathcal{D}_{\text{val}} = \{(X_i, Y_i)\}_{i \in \mathcal{I}_{\text{val}}}$$

2.2.7.1. Validación cruzada

Cuando los datos son limitados, es fundamental dividirlos eficientemente entre los conjuntos de entrenamiento y validación. Sin embargo, esto puede resultar problemático: si se destina una mayor proporción de datos al entrenamiento, el conjunto de validación podría no ser representativo, capturando solo una pequeña muestra. Por otro lado, destinar más datos a la validación significa menos datos para entrenar, lo que podría llevar al subajuste o sobreajuste del modelo.

La validación cruzada es una técnica que optimiza el uso de datos en situaciones de limitación, permitiendo que cada dato etiquetado se utilice tanto para entrenamiento como

para validación. En este proceso, el conjunto de datos se divide en k particiones (o *folds*) de tamaño similar, siendo comúnmente $k = 5$ o $k = 10$. El modelo se entrena k veces, usando en cada ciclo $k-1$ particiones para el entrenamiento y la partición restante para la validación. Así, cada partición se usa como conjunto de validación una vez, y los resultados obtenidos se promedian para obtener una estimación más precisa del rendimiento en datos no vistos.

Estas k particiones se denotan como $\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_k$, donde cada $\mathcal{D}_t$ representa uno de los subconjuntos de validación generados en el proceso de validación cruzada. Para cada iteración t, el conjunto de entrenamiento $\mathcal{D}_{\text{ent}}^{(t)}$ se obtiene excluyendo la partición de validación $\mathcal{D}_t$ del conjunto total $\mathcal{D}_{\text{ent}}$, es decir:

$$\mathcal{D}_{\text{ent}}^{(t)} = \mathcal{D}_{\text{ent}} \setminus \mathcal{D}_t, \quad \mathcal{D}_{\text{val}}^{(t)} = \mathcal{D}_t$$

Para cada uno de estos *folds*, el modelo se entrena en el conjunto de entrenamiento $\mathcal{D}_{\text{ent}}^{(t)}$ y se valida utilizando el conjunto de validación $\mathcal{D}_{\text{val}}^{(t)}$. La función de pérdida utilizada para ajustar los parámetros del modelo es:

$$\hat{\theta}_t = \arg\min_{\theta} \sum_{(X_i, Y_i) \in \mathcal{D}_{\text{ent}}^{(t)}} L\big(Y_i, \hat{f}_\theta(X_i)\big)$$

Finalmente, se calcula una métrica promedio tomando la media de las métricas obtenidas de cada iteración de validación. Este promedio proporciona una estimación más robusta del rendimiento del modelo en datos no vistos.

2.2.7.2. Ajuste de hiperparámetros

Con el objetivo de buscar la mejor configuración de hiperparámetros, existen diversos enfoques, entre los que se destacan la búsqueda en rejilla (*grid search*) y la búsqueda aleatoria (*random search*).

El primero consiste en seleccionar un conjunto de hiperparámetros, y una serie de valores para cada uno de ellos. A continuación, se ajusta un modelo por cada combinación posible de estos valores y se evalúa cada uno de ellos en el conjunto de validación. Finalmente, se obtiene la configuración de hiperparámetros que mayor rendimiento haya obtenido.

El segundo, por el contrario, en vez de ajustar un modelo por cada posible combinación, solamente se elige un número determinado de ellos. Este enfoque resulta computacionalmente menos costoso. No obstante, puede que no obtenga la mejor configuración de hiperparámetros.

2.2.8. Normalización y estandarización

Como se verá en el capítulo 3 existen algunas técnicas que se benefician de que los datos de entrada sigan una forma determinada.

2.2.8.1. Normalización

La normalización convierte los datos al rango $[0, 1]$ mediante la siguiente transformación:

$$X_{ij_{\text{norm}}} = \frac{X_{ij} - X_{ij_{\text{mín}}}}{X_{ij_{\text{máx}}} - X_{ij_{\text{mín}}}} \tag{2.1}$$

donde:

- X_{ij} es el valor original,
- $X_{ij_{\text{mín}}}$ es el valor mínimo del conjunto de datos,

- $X_{ij_{\text{mín}}}$ es el valor máximo del conjunto de datos.

2.2.8.2. Estandarización

La estandarización transforma los datos para que tengan media 0 y desviación estándar 1:

$$x_{ij_{\text{std}}} = \frac{X_{ij} - \mu}{\sigma} \tag{2.2}$$

donde:

- X_{ij} es el valor original,
- μ es la media del conjunto de datos,
- σ es la desviación estándar del conjunto de datos.

2.2.9. Métricas

Para evaluar el rendimiento de un modelo, se requieren criterios objetivos, y es aquí donde intervienen las métricas. Las métricas son unos criterios que cuantifican la capacidad predictiva de un procedimiento, y son ajenas al mismo. A diferencia de la función de pérdida, que toma un papel activo en el ajuste del procedimiento, las métricas no lo hacen, sino que se emplean una vez ya se ha ajustado el procedimiento. Existen métricas específicas para problemas de clasificación y otras para problemas de regresión [16]. Además, dependiendo de si se trata de problemas binarios o multiclase, se puede hacer otra distinción de métricas. En este caso se mostrarán las métricas para el caso binario.

- **Matriz de Confusión**: Tabla que muestra el rendimiento del modelo de clasificación al comparar las predicciones con los valores reales.

	Predicción: Positivo	Predicción: Negativo
Real: Positivo	TP	FN
Real: Negativo	FP	TN

Tabla 2.1: Matriz de confusión

  donde:

  - **TP** (verdaderos ositivos) son los casos correctamente predichos como positivos.
  - **TN** (verdaderos negativos) son los casos correctamente predichos como negativos.
  - **FP** (falsos positivos) son los casos predichos como positivos pero que son negativos.
  - **FN** (falsos negativos) son los casos predichos como negativos pero que son positivos.

- **Exactitud(Accuracy)**:

$$\text{Exactitud} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}.$$

  Mide la proporción de predicciones correctas sobre el total de predicciones. Se puede entender como una estimación de la probabilidad de que la predicción sea correcta. Esta última interpretación puede ser aplicada a las siguientes métricas.

No obstante, puede que esta métrica no sea útil en ciertos casos. Por ejemplo, si se quiere predecir si un paciente va a desarrollar una enfermedad, en caso de que solamente el 1 % de los ejemplos fuera positivo, si la predicción en todos los casos fuera negativa independientemente de los atributos de los datos, la tasa de predicciones correctas sería del 99 %, pero es evidente que el procedimiento carecería de poder predictivo alguno. Es por ello que en función del caso se emplean otras métricas más representativas del poder predictivo real.

- **Precisión (Precision)**:

$$\text{Precisión} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Mide la proporción de verdaderos positivos sobre el total de positivos predichos. Esta métrica es particularmente útil cuando los falsos positivos son costosos. Por ejemplo, si un tratamiento es caro y conlleva altos riesgos, es esencial minimizar la tasa de falsos positivos para evitar gastos innecesarios y posibles perjuicios para el paciente.

- **Exhaustividad (Recall)**:

$$\text{Exhaustividad} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Mide la proporción de verdaderos positivos sobre el total de casos realmente positivos. Esta métrica es especialmente valiosa cuando los falsos negativos son costosos. Por ejemplo, en un control de seguridad aeroportuaria, no detectar un explosivo podría tener consecuencias graves. En este contexto, es crucial minimizar la tasa de falsos negativos.

- **Especificidad**:

$$\text{Especificidad} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

Mide la proporción de verdaderos negativos sobre el total de casos realmente negativos. Esta métrica es especialmente valiosa cuando los falsos positivos son costosos.

- **F1 Score**:

$$F1 = 2 \cdot \frac{\text{Precisión} \cdot \text{Exhaustividad}}{\text{Precisión} + \text{Exhaustividad}}$$

Es la media armónica entre precisión y exhaustividad, útil en situaciones con desbalance en las clases.

- **Tasa de Falsos Positivos (FPR):**

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

Mide la proporción de falsos positivos sobre el total de casos realmente negativos. Esta resulta especialmente relevante cuando los falsos positivos son costosos.

- **Curva ROC (Receiver Operating Characteristic)**:

Gráfica que muestra la tasa de verdaderos positivos (TPR o exhaustividad) frente a la tasa de falsos positivos (FPR) a diferentes umbrales de clasificación u, tal y como se ha presentado en la sección 2.2.4.1. La curva ROC (ver figura 2.1) se genera calculando los pares $(exhaustividad, 1 - especificidad)$, es decir, $(exhaustividad, FPR)$ para una sucesión de umbrales $u \in [0, 1]$. En ocasiones, ese umbral será más permisivo y hará que más ejemplos sean clasificados como positivos, mientras que en otras ocasiones, el ejemplo tendrá que tener muchas probabilidades de ser positivo para que se clasifique como tal.

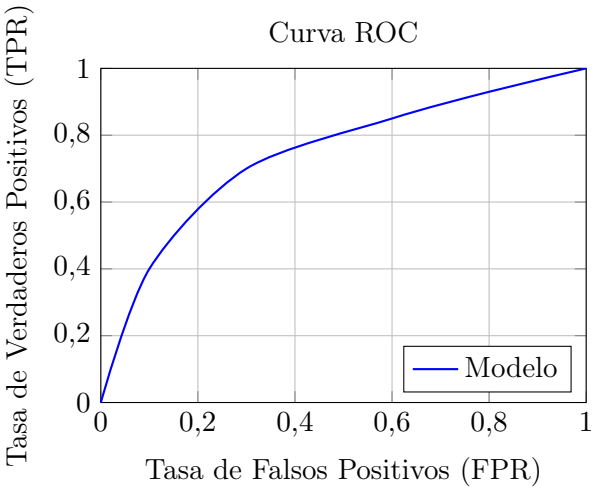

Figura 2.1: Curva ROC del modelo.

- **AUC (Área Under the Curve)**: Es el área debajo de la curva ROC. Mide la capacidad del modelo para distinguir entre clases. Un valor de 1 indica una perfecta distinción, mientras que un valor de 0,5 indica un modelo aleatorio (ver figura 2.2).

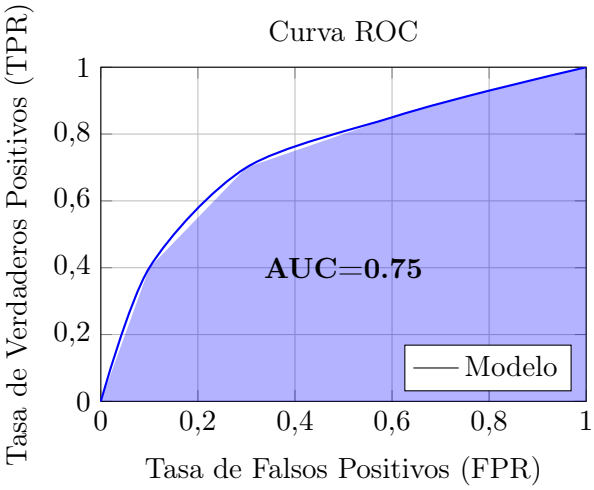

Figura 2.2: Área bajo la curva ROC (AUC).

La selección de métricas depende del problema específico y de los objetivos del modelo. Como ya se ha mostrado, puede que en ciertos problemas sea más conveniente usar ciertas métricas. Además, el propio conjunto de datos del que se dispone y la naturaleza del mismo también pueden determinar qué métricas resultan más convenientes.

Capítulo 3

Técnicas

Esta sección tiene como objetivo presentar y detallar una serie de técnicas que posteriormente servirán como base para desarrollar el estudio práctico.

3.1. K-NN

A lo largo de esta sección se hará uso del manual [15]. El método de los k vecinos más cercanos (del inglés *k-Nearest Neighbors*, abreviado *k-NN*) es un procedimiento basado en memoria que no requiere de ninguna fase de entrenamiento.

3.1.1. Introducción y fundamentos

Resulta probable que dos instancias cuyos atributos tienen valores parecidos pertenezcan a la misma clase. Por tanto, parece razonable mirar cuál es la clase mayoritaria entre las instancias más parecidas a un dato que se quiera clasificar. Además, puede ser que la instancia más cercana a un dato sea de una clase y que, por ejemplo, las siguientes nueve pertenezcan a otra. En este caso resultaría evidente que no solamente se debe tener en cuenta la instancia más cercana, sino que puede resultar mejor tomar las k instancias más parecidas para llevar a cabo la clasificación.

Teniendo esta idea intuitiva en mente, el funcionamiento del método es el siguiente: en primer lugar, se almacena un conjunto de datos, que corresponde al conjunto de entrenamiento $\mathcal{D}_{\text{ent}}$. Posteriormente, dado un dato de entrada X_q, se buscan los k puntos más cercanos a X_q dentro del conjunto de entrenamiento, y se asigna la clase mayoritaria entre esos k vecinos. Cabe destacar que el conjunto que se almacena es el de entrenamiento para que luego se pueda probar el rendimiento con el conjunto de prueba. No obstante, una vez elegido el mejor modelo, se usan todos los datos para ajustar el modelo. Esta práctica será recurrente en las demás técnicas descritas:

$$\hat{f}_\theta(X_q) = \arg\operatorname*{máx}_{c \in \mathcal{C}} \sum_{i=1}^{k} w_i [c = Y_i] \tag{3.1}$$

donde:

$$[c = Y_i] = \begin{cases} 1, & \text{si } c = Y_i, \\ 0, & \text{en otro caso.} \end{cases} \tag{3.2}$$

Parece razonable que aquellas instancias más cercanas al dato que se quiera clasificar tomen un peso más importante en la predicción de la clase. Es por ello que se introduce el término w_i que indica si la clasificación se llevará a cabo atribuyendo una mayor importancia a las instancias más cercanas (ponderación por distancia), o si por el contrario, las k instancias más cercanas tomarán el mismo peso (ponderación uniforme).

$$w_i = \begin{cases} \frac{1}{\text{distancia}(X_i, X_q)}, & \text{caso ponderado por distancia} \\ 1, & \text{en otro caso.} \end{cases} \tag{3.3}$$

Podría suceder que entre las k instancias más cercanas hubiera más de una clase mayoritaria, es decir, que ocurriera un empate entre más de una clase. Aunque en el caso de la ponderación por distancia resulte más difícil, también podría suceder lo mismo. Por tanto, en caso de existir un empate, un posible mecanismo sería elegir una de las clases mayoritarias al azar. No obstante, en el caso binario, si la ponderación es uniforme y k es impar, entonces es imposible que exista un empate, por lo que en este caso se tiende a usar una k de estas características.

3.1.1.1. Distancia y estandarización

Para medir la cercanía entre los puntos, se puede utilizar, entre otras, la distancia euclídea:

$$d_i = \|X_i - X_q\| = \sqrt{\sum_{j=1}^{p} (X_{ij} - X_{qj})^2} \tag{3.4}$$

o la distancia Manhattan:

$$d_i = \|X_i - X_q\|_1 = \sum_{j=1}^{p} |X_{ij} - X_{qj}| \tag{3.5}$$

Normalmente se opta por estandarizar los atributos para que tengan media cero y varianza uno, ya que pueden estar expresados en distintas unidades o con escalas diferentes. De no hacerlo, podría suceder que un atributo con valores nominales más grandes dominara la medida de distancia y degradara el rendimiento del clasificador.

3.1.2. Hiperparámetros

El valor de k puede cambiar por completo el rendimiento del clasificador. Si este es muy grande, puede suceder que el clasificador no sea capaz de captar detalles sutiles entre ciertas clases. Por el contrario, si k es muy pequeño, el clasificador podría ser sensible al ruido.

Como ya se ha presentado anteriormente, otros hiperparámetros de *K-NN* son la distancia utilizada (euclídea, Manhattan...) y el tipo de ponderación que se le da a los vecinos (ponderación uniforme o ponderación por distancia).

3.1.3. Ventajas e inconvenientes

Entre sus principales ventajas se puede considerar que no precisa de un proceso de entrenamiento al uso. Es decir, no precisa de ajustar ningún parámetro. Además, es una técnica con una alta flexibilidad y que no asume una forma específica de la distribución.

No obstante, presenta varios inconvenientes. En primer lugar, el coste de computación en la fase de predicción es alto ya que debe calcular la distancia de la entrada respecto de

todas las instancias almacenadas. Ligado a esto, cabe destacar que es necesario tener almacenado el conjunto de datos para poder hacer predicciones. Por tanto, cuando el conjunto es grande, implica un coste en memoria. Además, se precisa de normalizar o estandarizar los datos para que la capacidad predictora no se vea deteriorada. Asimismo, puede resultar que existan variables superfluas que estén distanciando dos datos que realmente son más parecidos. De la misma forma, podría suceder que existieran variables redundantes que tuvieran el efecto contrario, es decir, que ciertos datos se estuvieran tomando por más parecidos de lo que realmente lo son.

3.2. Árboles de decisión

A lo largo de esta subsección se hará una descripción de los árboles de decisión. Además, se describirán los hiperparámetros principales que usa, así como las principales ventajas y desventajas que presenta [15].

3.2.1. Introducción y fundamentos

Los árboles de decisión realizan una partición recursiva del espacio de características, buscando regiones homogéneas respecto de la composición de la clase. Es decir, se busca hacer predicciones locales en función de ejemplos cercanos, así como sucedía en *k-NN*. Así pues, se parte de todo el espacio de características que se identifica como un nodo raíz. Este nodo contiene todos los datos de entrenamiento. Posteriormente, se hace una divisón de este espacio en dos regiones, cada una de las cuales se identifica con un nodo hijo de la raíz. Este proceso de divisón se itera recursivamente para cada nodo. De esta forma, las hojas son los nodos terminales, es decir, regiones que no se subdividen. Una rama es un camino desde el nodo raíz hasta una hoja. A cada región resultante de la partición se le asigna una clase, y todos los datos que estén contenidos en una misma región tomarán la misma predicción, que coincidirá con la clase mayoritaria dentro de la región.

Dado un nodo m, que representa una región R_m con N_m observaciones, sea

$$\hat{p}_{mc} = \frac{1}{N_m} \sum_{X_i \in R_m} [c = Y_i] \tag{3.6}$$

la proporción de observaciones de la clase c en el nodo m. Se clasifican las observaciones en el nodo m como

$$\hat{f}_\theta(X) = \arg\max_{c \in \mathcal{C}} \hat{p}_{mc} \tag{3.7}$$

Para encontrar la mejor partición en un árbol de clasificación, se sigue un proceso iterativo y voraz que busca minimizar la impureza o hetereogeneidad de los nodos. El objetivo es encontrar la división del espacio de atributos que minimice la impureza, es decir, que maximice la reducción de la impureza.

3.2.1.1. Impureza del Nodo

Para medir la impureza de un nodo m existen distintos criterios como la entropía, el índice de Gini o el error de clasificación:

- Entropía cruzada:

$$Q_m = -\sum_{c \in \mathcal{C}} \hat{p}_{mc} \log(\hat{p}_{mc}) \tag{3.8}$$

- Error de clasificación:

$$Q_m = \frac{1}{N_m} \sum_{X_i \in R_m} [Y_i \neq \hat{f}_\theta(X_i)] = 1 - \hat{p}_{m\hat{f}_\theta(X_i)} \tag{3.9}$$

- Índice de Gini:

$$Q_m = \sum_{c \neq c'} \hat{p}_{mc}\hat{p}_{mc'} = \sum_{c \in \mathcal{C}} \hat{p}_{mc}(1 - \hat{p}_{mc}) \tag{3.10}$$

3.2.1.2. División del Nodo

Para cada atributo j y para cada punto de división s, el nodo m se divide en dos nodos hijos:

- Nodo izquierdo (m_{izq}): $R_{izq}(j, s) = \{X_i \mid X_{ij} \leq s\}$, con N_{izq} observaciones.
- Nodo derecho (m_{der}): $R_{der}(j, s) = \{X_i \mid X_{ij} > s\}$, con N_{der} observaciones.

La impureza total en los nodos hijos se calcula mediante la media ponderada de las impurezas:

$$Q(j, s) = \frac{N_{izq}}{N_m} Q_{izq}(j, s) + \frac{N_{der}}{N_m} Q_{der}(j, s) \tag{3.11}$$

donde $Q_{izq}(j, s)$ y $Q_{der}(j, s)$ son las impurezas de los nodos m_{izq} y m_{der}, respectivamente.

3.2.1.3. Reducción de la impureza

Al realizar una divisón de un nodo en dos hijos, la reducción de la impureza se define como la diferencia entre la impureza del nodo padre Q_m y la impureza total de los nodos hijos $Q(j, s)$:

$$\Delta Q(j, s) = Q_m - Q(j, s) \tag{3.12}$$

El objetivo es maximizar $\Delta Q(j, s)$, o equivalentemente, minimizar $Q(j, s)$, para encontrar el par óptimo (j^*, s^*):

$$(j^*, s^*) = \arg\operatorname*{máx}_{j,s} \Delta Q(j, s) = \arg\operatorname*{mín}_{j,s} Q(j, s).$$

De esta forma, se calculan las impurezas de todas las combinaciones posibles de variables j y puntos de corte s para cada nodo, y se selecciona la división que lleve a la menor impureza total.

3.2.1.4. Proceso iterativo y poda

El proceso recién explicado se repite para cada nodo hijo recursivamente hasta que se cumpla alguna de las condiciones de parada, que pueden ser las siguientes:

- Alcanzar un número mínimo de observaciones en un nodo $N_{\min}$.
- Llegar a una profundidad máxima del árbol $d_{\max}$.
- No encontrar una división que mejore la impureza más de un umbral $\Delta Q_{\min}$.

El tamaño del árbol determina la complejidad del modelo. Por ello habitualmente se precisa de un mecanismo de control del tamaño del mismo, que se conoce como poda. Un posible enfoque sería dividir los nodos del árbol siempre y cuando la reducción de la impureza supera un umbral preestablecido. Esta estrategia podría resultar demasiado cortoplacista, ya que una reducción aparentemente nimia, podría conducir a una buena división posteriormente. Para evitar esto, una estrategia alternativa es construir un árbol grande T_0, deteniendo el proceso de división sólo cuando se alcanza algún tamaño mínimo de nodo. Luego, este árbol se poda usando el criterio de costo-complejidad que se explica a continuación.

Un subárbol $T \subset T_0$ es cualquier árbol que se pueda obtener podando T_0, es decir, colapsando cualquier número de sus nodos internos (no terminales). Sea $|T|$ el número de nodos terminales en T. Se define el criterio de costo-complejidad

$$C_\alpha(T) = \sum_{m=1}^{|T|} N_m Q_m + \alpha |T| \tag{3.13}$$

La idea es encontrar el subárbol $T_\alpha \subseteq T_0$ que minimice $C_\alpha(T)$. El hiperparámetro $\alpha \geq 0$ determina el equilibrio entre el tamaño del árbol y su capacidad de ajuste a los datos. Se trata de una regularización, como se explica en la sección 2.2.6.1. Valores grandes de α resultan en árboles más pequeños T_α, y viceversa.

3.2.2. Hiperparámetros

Como se ha ido presentando a lo largo de esta subsección, los árboles de decisión precisan de ciertos hiperparámetros que han de ser prefijados. Entre ellos destacan los siguientes:

- Criterio para medir impureza (entropía cruzada, error de clasificación o índice de Gini): en función del criterio elegido la homoegeneidad de cada región se medirá de una forma distina, lo que puede llevar a divisiones distintas de la región de atributos.
- Criterios de parada: prefijar los criterios de parada presentados en la sección 3.2.1.4 determina la forma y el tamaño que tomará el árbol, lo cual afecta directamente en las clasificaciones que se harán de los datos de entrada.

3.2.3. Ventajas e inconvenientes

La principal ventaja de los árboles de decisión es su alta interpretabilidad. A diferencia de la mayoría de técnicas existentes, los árboles de decisión no funcionan como una caja negra que recibe datos y devuelve resultados, sino que permiten entender por qué ha hecho cada clasificación. Además, permiten capturar relaciones complejas y no lineales entre las variables de entrada. También cabe destacar que son útiles para realizar selección de variables. Asimismo, son robustos ante datos atípicos (*outliers*) y pueden tratar datos faltantes (*missings*).

Entre las principales desventajas de esta técnica se puede destacar la alta sensibilidad a la variación de los datos de entrada, que puede llevar a crear árboles totalmente diferentes. Esto es lo que se conoce como varianza del modelo. Además, es sensible al desbalanceo de datos, haciendo que el árbol se incline hacia la clase mayoritaria.

3.3. Random Forest

En esta subsección se desarrollará la técnica de bosque aleatorio (del inglés *random forest*) que surge como extensión de los árboles de decisión presentados en la subsección anterior 3.2.

3.3.1. Introducción y fundamentos

Random Forest es un algoritmo de aprendizaje automático que, con el objetivo de mejorar la precisión de las predicciones, se basa en el principio de la agregación de modelos [15]. La idea detrás de esta técnica reside en crear un conjunto de árboles de decisión que, combinados entre sí adecuadamente, producen resultados más precisos.

3.3.1.1. Algoritmo

Para crear un bosque de árboles se sigue el siguiente procedimiento: en primer lugar, se generan varios subconjuntos de datos mediante el muestreo aleatorio con reemplazo, conocido como *bootstrap sampling*. Es decir, para cada subconjunto se eligen una cantidad de datos al azar y aquellos elegidos en un subconjunto pueden aparecer en otro. A continuación, en cada paso del entrenamiento de los árboles, se seleccionan aleatoriamente un número limitado de características para decidir la mejor división en cada nodo. Este proceso introduce diversidad entre los árboles y ayuda a reducir la correlación entre ellos, lo que contribuye a la estabilidad y precisión del modelo final.

Algoritmo de Random Forest:

1. Desde $b = 1$ hasta B (donde B es el número de árboles):
   a) Extraer una muestra bootstrap Z de tamaño N del conjunto de entrenamiento.
   b) Crear un árbol de decisión sobre el conjunto de datos bootstrap Z:
      1) Para cada nodo n sin dividir, se repiten los siguientes pasos hasta que se alcance el criterio de parada seleccionado:
         a' : Seleccionar aleatoriamente m variables de las p variables disponibles.
         b' : Elegir la mejor variable y el punto de corte entre las m seleccionadas.
         c' : Dividir el nodo en dos nodos hijos.
2. Resultado final: el modelo Random Forest consiste en el conjunto de árboles $\{T_b\}_{b=1}^{B}$.

Es decir, se han creado B árboles de clasificación distintos, cada uno de los cuales ha sido construido sobre un subconjunto aleatorio de los datos. A su vez, para cada uno de esos árboles, en cada paso de creación de dos nodos hijos a partir de un nodo padre, se eligen m variables aleatoriamente entre las cuales se ha de hacer la divisón que maximice la reducción de la impureza que se ha explicado en la sección 3.2.1.3

En el caso de un problema de clasificación, la predicción para un nuevo dato X se obtiene utilizando la votación mayoritaria entre los árboles:

$$\hat{f}_\theta(X) = \arg\max_{c \in \mathcal{C}} \sum_{b=1}^{B} [\hat{f}_b(X) = c]$$

donde $\hat{f}_b(x)$ es la predicción del b-ésimo árbol para la entrada X.

3.3.2. Hiperparámetros

Los principales hiperparámetros del algoritmo Random Forest son los siguientes:

- Número de árboles (B): es el número de árboles en el bosque. Un valor más alto suele mejorar la precisión del modelo, pero aumenta el tiempo de computación.
- Número máximo de características ($m_{\max}$): es el número máximo de características seleccionadas aleatoriamente que se consideran en cada división del árbol. Usualmente, en los problemas de clasificación se establece como $\sqrt{p}$.

- Tamaño de la muestra bootstrap (N): es el tamaño de la muestra extraída aleatoriamente (con reemplazo) para entrenar cada árbol. Este tamaño suele ser igual al tamaño del conjunto de datos original, aunque puede ser modificado.

Estos hiperparámetros permiten ajustar el modelo para equilibrar el sesgo y la varianza, y controlar el rendimiento del algoritmo en términos de precisión y tiempo de computación.

3.3.3. Ventajas e inconvenientes

Principalmente esta técnica reduce la varianza y aumenta la precisión respecto de los árboles de decisión. Además, es capaz de manejar datos desbalanceados de manera más efectiva.

No obstante, se pierde la interpretabilidad que ofrecían los árboles de decisión debido a que se genera más de un árbol. También cabe destacar que el coste computacional aumenta respecto a los árboles de decisión por el mismo motivo.

3.4. Regresión logística

A lo largo de esta subsección se presentará la técnica de la regresión logística, para lo cual se hará uso del manual [15].

3.4.1. Introducción y fundamentos

La regresión logística es un modelo empleado para problemas de clasificación cuyo objetivo es modelar la probabilidad de que un dato de entrada pertenezca a una determinada clase. Se diferencia de la regresión lineal en tanto este último puede predecir valores fuera del intervalo $[0, 1]$. Se trata, pues, de un método que transforma un valor continuo que predice un modelo de regresión lineal a una probabilidad de pertenecer a la clase positiva c_1 o a la clase negativa c_2.

Se trata de un modelo global, al contrario de las téncicas expuestas anteiormente. Es decir, no tiene en cuenta sólamente los datos similares, sino que tiene en cuenta la totalidad de los datos para predecir la clase a la que pertenece un dato.

3.4.1.1. Formulación del Modelo

Para un problema con 2 clases, el modelo de regresión logística define una función logit (logaritmo de razón de probabilidad):

$$\log \frac{\Pr(Y = c_1 | X = x)}{\Pr(Y = c_2 | X = x)} = \beta_{10} + \beta_1^T x \tag{3.14}$$

$$\tag{3.15}$$

Donde:

- $\beta_{10} \in R$ el término independiente para la clase c_1
- $\beta_1 \in R^p$ es el vector de coeficientes para la clase c_1
- $\beta = \{\beta_{10}, \beta_1\}$

Si se despejan las probabilidades, se tiene:

$$p(x_1;\beta) = Pr(Y = c_1|X = x) = \frac{\exp(\beta_{10} + \beta_1^T x)}{1 + \exp(\beta_{10} + \beta_1^T x)} \tag{3.16}$$

$$p(x_2;\beta) = Pr(Y = c_2|X = x) = 1 - Pr(Y = c_1|X = x) = \frac{1}{1 + \exp(\beta_{10} + \beta_1^T x)} \tag{3.17}$$

Estas fórmulas garantizan que $\sum_{i=1}^{2} \Pr(Y = c_i|X = x) = 1$ y que cada probabilidad esté en el intervalo $[0, 1]$.

Cabe destacar que las *odds* representan la razón entre la probabilidad de que un dato pertenezca a la clase c_1 y la probabilidad de que pertenezca a la clase c_2.

$$\text{odds}(x) = \frac{p(x_1;\beta)}{p(x_2;\beta)} = \frac{\frac{\exp(\beta_{10}+\beta_1^T x)}{1+\exp(\beta_{10}+\beta_1^T x)}}{\frac{1}{1+\exp(\beta_{10}+\beta_1^T x)}} = \exp(\beta_{10} + \beta_1^T x)$$

El *odds ratio* indica cómo cambian las *odds* si se varía en una unidad el valor de una variable de entrada.

$$OR = \frac{\text{odds}(x^*)}{\text{odds}(x)} = \frac{\exp(\beta_i)\exp(\beta_0 + \beta_1^T x)}{\exp(\beta_0 + \beta_1^T x)} = \exp(\beta_i)$$

donde

$$x^* = (x_1, \ldots, x_i + 1, \ldots, x_p)$$

Si $\beta_i > 0$, el *odds ratio* es mayor que 1, lo que significa que un aumento en x_i aumenta las odds. Si $\beta_i < 0$, por el contrario, el *odds ratio* es menor que 1, lo cual indica que un aumento en x_i disminuye las odds.

3.4.1.2. Estimación por Máxima Verosimilitud

La distribución de Bernoulli describe una variable aleatoria X que puede tomar valores 0 o 1. Considerando que la probabilidad de que la variable tome el valor 1 es p y el valor 0 es $1 - p$, la función de masa de probabilidad de una variable aleatoria X con distribución Bernoulli es:

$$P(X = x) = p^x(1 - p)^{1-x}, \quad \text{donde } x \in \{0, 1\}.$$

Asumiendo N observaciones independientes (X_i, Y_i) que siguen la distribución de Bernoulli, la probabilidad conjunta de obtener esos datos es:

$$P(X_1 = x_1, X_2 = x_2, \ldots, X_n = x_n) = \prod_{i=1}^{n} p^{x_i}(1 - p)^{1-x_i}$$

Para estimar los parámetros óptimos β, se utiliza el método de máxima verosimilitud, que consiste en maximizar la función de verosimilitud. Esta función calcula la probabilidad de obtener los datos observados en función de la configuración de parámetros β. Asumiendo N observaciones independientes (X_i, Y_i) que siguen la distribución de Bernoulli, la función de verosimilitud es:

$$L(\beta) = \prod_{i=1}^{N} p(x_i;\beta)^{Y_i}(1 - p(x_i;\beta))^{1-Y_i} \tag{3.18}$$

donde $Y_i = 1$ para la clase positiva c_1 y $Y_i = 0$ para la clase negativa c_2.

Para facilitar los cálculos, se trabaja con el logaritmo de la verosimilitud:

$$\ell(\beta) = \log L(\beta) \tag{3.19}$$

$$= \sum_{i=1}^{N} [Y_i \log p(x_i;\beta) + (1 - Y_i)\log(1 - p(x_i;\beta))] \tag{3.20}$$

$$= \sum_{i=1}^{N} \left[Y_i(\beta_0 + \beta^T x_i) - \log(1 + e^{\beta_0 + \beta^T x_i})\right] \tag{3.21}$$

Para encontrar el máximo de la log-verosimilitud, se deriva con respecto a los parámetros y se iguala a cero:

$$\frac{\partial \ell(\beta)}{\partial \beta} = \sum_{i=1}^{N} x_i(Y_i - p(x_i;\beta)) = 0 \tag{3.22}$$

Estas son $p + 1$ ecuaciones no lineales respecto a β.

3.4.1.3. Algoritmos de Optimización

Como las ecuaciones anteriores son no lineales, una de las formas de resolverlas es iterativamente mediante el método de Newton-Raphson. El algoritmo actualiza los parámetros según:

$$\theta^{\text{nuevo}} = \theta^{\text{anterior}} - \left(\frac{\partial^2 \ell(\theta)}{\partial\theta\partial\theta^T}\right)^{-1} \frac{\partial \ell(\theta)}{\partial \theta} \tag{3.23}$$

donde las derivadas se evalúan en θ^{anterior}.

3.4.2. Hiperparámetros

El principal hiperparámetro específico de este método es el algoritmo de optimización para maximizar la función de verosimilitud.

Además, para evitar el sobreajuste, en la regresión logística se tiende a emplear un término de regularización en la función log-verosimilitud que penaliza los valores altos de β, tal y como se explica en la sección 2.2.6.1

3.4.3. Ventajas e inconvenientes

Uno de los puntos positivos de esta técnica es la simpleza de la misma, así como la interpretabilidad de los resultados. Además, como se ha explicado en la sección 3.4.1.1, la regresión logística permite saber cómo afecta la variación de un parámetro en la probabilidad de pertenecer a una clase respecto otra, lo cual puede resultar muy conveniente.

Sin embargo, como la probabilidad de que un cierto dato pertenezca a una clase es monótona sobre la combinación lineal de los datos de entrada, puede no ajustarse bien a los datos. Además, si las clases no están balanceadas, las predicciones pueden verse sesgadas hacia la clase mayoritaria. Asimismo, es necesario tratar *outliers* y *missings*.

3.5. Redes Neuronales

En esta subsección se describirá el funcionamiento de las redes neuronales y se basará en [15].

3.5.1. Introducción y fundamentos

Las redes neuronales están inspiradas en el funcionamiento de las neuronas biológicas, y en la red que éstas forman. Se trata de un modelo global y que tiene mucha flexibilidad para ajustar patrones complejos, para lo cual potencialmente usa muchos parámetros.

Las redes neuronales están compuestas por neuronas, que se organizan en capas. Cada neurona recibe un vector de valores de la capa anterior y lo multiplica por un vector de pesos. A este producto se le suma un término independiente y se le aplica una función σ, que se conoce como función de activación. Este resultado se lleva a la siguiente capa, y así sucesivamente.

Existen diferentes tipos de redes neuronales en función de su configuración y su forma de proceder, pero a continuación se describirá una red *feedforward*.

En primer lugar existe una capa de entrada que tiene tantas neuronas como variables en el conjunto de datos. Cada una de ellas está conectada a todas las neuronas de la siguiente capa. En esta segunda capa, conocida como capa intermedia, cada neurona toma todos los valores que recibe de las neuronas de la capa anterior y los multiplica por un vector de pesos. Adicionalmente se le suma un término independiente y a todo ello se le aplica la función de activación, dando como resultado lo que se conoce como una característica derivada z_{ln} donde l denota el número de la capa intermedia y n el nodo de dicha capa. Este resultado es enviado a la siguiente capa (que puede ser otra capa intermedia o la capa de salida). Puede haber tantas capas intermedias L como se quieran, y una se conecta a la siguiente en la forma que se acaba de detallar. Cabe destacar que en cada capa intermedia puede haber un número de neuronas variable N.

Finalmente, los resultados producidos por las neuronas de la última capa intermedia se conectan con las neuronas de la capa de salida, donde hay tantas neuronas como clases. Estas neuronas proceden de la misma forma que las neuronas intermedias. No obstante, una vez todas han generado un resultado, se aplica una función g_c que sirve para que los resultados calculados por la red se puedan interpretar como resultados del problema.

$$Z_{1n} = \sigma(\alpha_{01n} + \alpha_{1n}^T X), \quad n = 1, \ldots, N_l \tag{3.24}$$

$$Z_{ln} = \sigma(\alpha_{0(l-1)n} + \alpha_{(l-1)n}^T Z_l), \quad n = 1, \ldots, N_l \quad l = 2, \ldots, L \tag{3.25}$$

$$T_c = \beta_{0c} + \beta_c^T Z_L, \quad c = 1, \ldots, k \tag{3.26}$$

$$g_c(T), \quad c = 1, \ldots, k \tag{3.27}$$

donde $Z_l = (Z_1, Z_2, \ldots, Z_{N_l})$ y $T = (T_1, T_2, \ldots, T_k)$.

Aunque existan múltiples opciones para elegir la función de activación, habitualmente se elige la función sigmoide:

$$\sigma(v) = \frac{1}{1 + e^{-v}} \tag{3.28}$$

La función g_c hace una transformación final y da como resultado el vector de salidas. Aunque en ocasiones se usa la función *argmax* (que toma la clase cuyo valor de T es mayor),

como no resulta conveniente para el ajuste de la red, habitualmente se adopta la función *softmax*:

$$g_c(T) = \frac{e^{T_c}}{\sum_{l=1}^{c} e^{T_l}} \tag{3.29}$$

Esta función de transformación atribuirá a cada clase una probabilidad. En el caso binario, se puede configurar la capa de salida para que solamente tenga una neurona, la cual se encargará de calcular la probabilidad de que el dato de entrada pertenezca a la clase positiva.

El modelo presentado tiene parámetros desconocidos, a menudo llamados pesos. El objetivo reside en encontrar valores para dichos pesos que hagan que el modelo se ajuste lo mejor posible a los datos de entrenamiento. Se denotará al conjunto completo de pesos por γ, que consiste en:

- $\{\alpha_{0ln}, \alpha_l n;\ l = 1, 2, \ \ldots, L \quad n = 1, \ldots, N_l\}$, un total de $N_1 * (p+1) + \sum_{l=2}^{L} (N_l) * (N_{l-1})$ pesos.
- $\{\beta_{0c}, \beta_c;\ c = 1, 2, \ldots, k\}$, un total de $k(N_L + 1)$ pesos.

El clasificador correspondiente es

$$\hat{f}_\gamma(X) = \arg\max_c g_c(X)$$

y se puede usar la entropía cruzada como función de error:

$$R(\gamma) = -\sum_{i=1}^{n} \sum_{c \in \mathcal{C}} [Y_i = c] \log(g_c(X_i)) \tag{3.30}$$

donde $g_c(X_i)$ denota la probabilidad que devuelve la red de que X_i pertenezca a la clase c.

Para ajustar el modelo se debe minimizar el error $R(\gamma)$. Para ello, el enfoque habitual es el descenso de gradiente, que en este contexto se conoce como *back-propagation*.

3.5.1.1. Back-propagation

La idea principal de esta técnica es calcular la derivada de la función de pérdida $R(\gamma)$ respecto de todos los pesos γ usando la regla de la cadena. De esta forma, se actualizan los nuevos valores de los pesos según la siguiente ecuación y se vuelve a iterar de la misma forma hasta que la diferencia de un paso sea menor de un umbral o hasta que se completa un número máximo de etapas (también conocidas como épocas):

$$\gamma^{(t+1)} = \gamma^{(t)} - \eta \frac{\partial R}{\partial \gamma}$$

donde η es la tasa de aprendizaje y $\gamma^{(t)}$ indica el valor de γ en la época t de *back-propagation*.

3.5.2. Hiperparámetros

Los principales hiperparámetros de este modelo son los siguientes: por un lado el número de capas intermedias L y el número de neuronas por cada capa N_l determinan la forma que va a tener la red neuronal, y por tanto, la capacidad que va a tener para ajustarse a los datos.

Por otro lado, la función de activación σ determinará qué función se va a modificar para tratar de ajustarla lo máximo posible a los datos.

Cabe mencionar la tasa de aprendizaje del modelo, puesto que una tasa muy grande podría hacer que no se llegara al mínimo de la función de pérdida, mientras que un valor muy pequeño podría ralentizar mucho el proceso.

Unido a esto, se debe mencionar el número máximo de épocas del modelo, que determina cuántas veces se puede hacer este proceso de *back-propagation*.

Finalmente, otro hiperparámetro a tener en cuenta es la inicialización de los pesos, que habitualmente se inicializan con valores aleatorios (a excepción de los términos independientes que se inicializan a cero). En caso de inicializar todos los pesos a 0, podría suceder que el gradiente se anulara y que por tanto no se actualizaran los pesos mediante *back-propagation*.

3.5.3. Ventajas e inconvenientes

Como una de las características más positivas de las redes neuronales es su capacidad de aproximar funciones complejas. Además, se pueden aplicar para datos de entrada de todo tipo, como texto, imágenes o vídeos.

No obstante, habitualmente requieren de un volúmen de datos muy grande para que puedan funcionar correctamente, y es por eso que en ciertos casos, otros métodos más clásicos resultan mejores para algunas tareas. Además, la potencia de cómputo y de memoria requerida puede ser muy grande debido al gran número de pesos que debe ajustar.

Capítulo 4

Estudio Práctico

En esta sección se detallará cómo se ha abordado el problema práctico y se presentarán los resultados asociados.

4.1. Configuración

A continuación se describirá en profundidad cómo se ha realizado el estudio práctico. El código que se ha seguido para llevarlo a cabo se encuentra en el apéndice B

4.1.1. Herramientas utilizadas

Para el desarrollo del estudio práctico se ha utilizado el lenguaje de programación *python*, así como algunas librerías, entre las que se pueden destacar *pandas*, *numpy* y *scikitlearn*. Además, como se ha detallado en la sección 2.1.3, se ha hecho uso de la plataforma *kaggle* para obtener el conjunto de datos.

4.1.2. Preprocesamiento de los datos

Antes de poder usar los datos para ajustar los modelos, se deben hacer una serie de modificaciones para que este ajuste se haga de la forma más óptima posible. De lo contrario, podría suceder que las técnicas no tuvieran un rendimiento esperado.

Para ello, estos han sido los pasos que se han seguido para preprocesar el conjunto de datos que se ha presentado en la sección 2.1.3.

4.1.2.1. Eliminación de valores nulos

En primer lugar, se ha examinado si había valores nulos en el conjunto de datos, puesto que para todas las técnicas presentadas en el capítulo 3 precisan de no tener valores faltantes. En este caso, no había ningún valor nulo, por lo que no se ha precisado de modificar el conjunto de datos. No obstante, de haberlos, se hubieran eliminado todas las filas con al menos un valor nulo en alguna de sus columnas. De esta forma, aunque la cantidad de datos se hubiera reducido, se garantizaría la existencia de todos los valores.

4.1.2.2. Eliminación de datos repetidos

A continuación, se ha analizado si había datos repetidos. En este caso, tampoco los había. Sin embargo, resulta importante eliminar valores duplicados en caso de que los haya, puesto que pueden añadir sesgo en los resultados que se obtengan.

4.1.2.3. Desbalance de clases

Se ha observado que el número de reservas que han resultado en cancelación es la mitad de aquellas que no se han cancelado. Es decir, las clases están moderadamente desbalanceadas. No obstante, como no es un desbalanceo extremo y como hay suficientes datos, no se ha optado por balancear clases. Para ello, se podrían haber eliminado algunos datos de la clase mayoritaria (submuestreo) o se podrían haber creado datos sintéticos de la clase minoritaria (sobremuestreo).

4.1.2.4. Variables derivadas

En aras de intentar mejorar la capacidad predictiva del clasificador, se pueden crear variables nuevas a partir de variables existentes. En este caso, como existe una variable $no_of_previous_cancellations$ (que se denotará por $cancellations$) que mide el número de cancelaciones previas y $no_of_previous_bookings_not_canceled$ (que se denotará por $bookings$) que mide el número de reservas previas no canceladas, se puede definir una nueva variable que mida la proporción de reservas previas canceladas. De esta forma, las técnicas que se aplicarán para crear los clasificadores podrán, potencialmente, sacar más partido de esta nueva variable (que se llamará $proporcin_cancelaciones$) que de las dos mencionadas de forma separada. Para ello, para cada dato, se seguirá esta fórmula para calcular el valor de la nueva variable:

$$proporcin_cancelaciones = \begin{cases} 0, & \text{si } cancellations = 0 \text{ y } bookings = 0 \\ \frac{cancellations}{cancellations+bookings}, & \text{en caso contrario} \end{cases}$$

4.1.2.5. Correlación de las variables

Como hay una cantidad considerable de variables, podría ser beneficioso si se pudieran eliminar aquéllas que no aporten mucha información. Una de las formas para lograr esto es analizar qué correlación existe entre diferentes variables. Si dos variables están muy correlacionadas, se podría eliminar una variable y apenas se perdería información.

En este conjunto de datos no existen correlaciones más allá del 0,5 en valor absoluto, por lo que no se deben eliminar variables. Además, a excepción de unos pocos pares de variables que tienen valores cercanos a 0,5, la mayoría de las variables mantienen una correlación muy baja con las otras variables.

4.1.2.6. Variables categóricas

Como se ha comentado en la sección 2.1.3, existen ciertas variables categóricas, entre las cuales se encuentran $type_of_meal_plan$,$room_type_reserved$ y $market_segment_type$. Dado que en la documentación del conjunto de datos no aparecen los posibles valores que pueden tomar estos últimos, se han analizado todos los valores distintos que toman y se ha observado lo siguiente.

1. $type_of_meal_plan$ puede tomar los valores $MealPlan1$ y $MealPlan2$ y $NotSelected$.
2. $room_type_reserved$ puede tomar valores desde $Room_type1$ hasta $Room_type7$.
3. $market_segment_type$ puede tomar los valores $Offline$, $Online$, $Corporate$, $Aviation$ y $Complementary$.

Además, como no se ofrece información específica sobre los valores que pueden tomar estas variables, y como no se puede conocer si existe una ordenación lógica de estos valores, se ha optado por crear una columna por cada uno de los valores que pueden tomar estas variables.

Estas columnas se sustituyen por las columnas originales de las variables y solamente toman valores de verdadero o falso indicando que un dato toma un valor específico de cada una de las variables. Esto se ha implementado mediante *one hot encoding*.

4.1.2.7. Outliers

Se ha analizado si existen valores atípicos en los datos (ver apéndice A). Se ha observado que, sobre todo en no_of_adults, en torno a un 30 % de los datos son *outliers*. Sin embargo, cabe destacar que se tratan de valores plausibles de la variable que simplemente son diferentes al resto de la distribución. No obstante, en el resto de atributos hay pocos datos atípicos. Ante esto, no se ha decidido hacer ningún cambio en el conjunto de datos dado el número reducido de los *outliers*.

4.1.2.8. Normalización y estandarización

Como algunas técnicas seleccionadas pueden verse potenciadas por la estandarización (ver sección 2.2.8.2) o normalización (ver sección 2.2.8.1) de las variables, como se ha mencionado en el apartado 3.4, se ha optado por el siguiente enfoque. Las variables numéricas se han estandarizado y normalizado con la intención de probar si existe una gran diferencia entre los dos enfoques. Además, la variable $arrival_year$ se ha transformado a 0 si toma el valor 2017 y a 1 si toma el valor 2018, ya que solamente toma esos dos valores. Además, con la variable a predecir *booking_status* se ha hecho algo similar. En caso de que la reserva se haya cancelado, se ha transformado a 0 y en caso de que no, se ha convertido en 1.

4.1.3. División de los datos

Como se ha explicado en la sección 2.2.5, es necesario dividir los datos en dos conjuntos, entrenamiento y prueba. En este caso, se ha optado por que la división sea del 80 % para entrenamiento y 20 % para prueba. Además, como existe cierto desbalanceo de clases, se ha optado por usar el muestreo estratificado tanto para el conjunto de entrenamiento como para el conjunto de prueba. De esta forma, en ambos conjuntos se mantendrá la proporción que existe de instancias positivas y negativas.

4.1.4. Ajuste de hiperparámetros

Tal y como se ha detallado en la sección 2.2.7.2, con el objetivo de ajustar los hiperparámetros para conseguir un mejor rendimiento, se precisa de una estrategia, ya que sería inabordable probar todas las combinaciones posibles de los mismos, puesto que los valores que pueden tomar algunos hiperparámetros son, en muchos casos, infinitos. Por ello, en este caso se ha optado por usar una búsqueda en rejilla, que consiste en elegir un subconjunto finito de valores de los hiperparámetros y evaluar todas las posibles combinaciones entre ellos.

Además, la implementación de la búsqueda en rejilla incluye el proceso de validación cruzada explicada en la sección 2.2.7.1. En este caso, se ha optado por hacer 5 $folds$. Cabe destacar que la métrica que se ha seleccionado para guiar el proceso de selección de modelos ha sido el $F1Score$.

4.1.5. Hiperparámetros

Los hiperparámetros con los que se ha hecho la búsqueda en rejilla han sido, en la medida de lo posible, los detallados durante el capítulo 3:

- KNN: k, tipo de ponderación (uniforme o por distancia) y distancia empleada (euclídea o Manhattan).

- Árbol de decisión: profundidad máxima del árbol, número mínimo de observaciones por hoja y constante de poda α.
- Random Forest: los mismos que en los árboles de decisión exceptuando la constante de poda, y adicionalmente el número de árboles y el número máximo de características.
- Regresión logística: inverso de la consante de regularización y un parámetros que controla el equilibrio entre la regularización $L1$ Y $L2$.
- Red neuronal: el número de capas, el número de neuronas por capa y la tasa de aprendizaje.

En la tabla 4.1 se muestran los rangos establecidos para la búsqueda de valores de los hiperparámetros.

Modelo	**Rango de hiperparámetros probados**
KNN	`n_neighbors` $\in$ {1, 3, 5, 11, 21, 31, 51, 101}, `weights` $\in$ { distance, uniform}, `metric` $\in$ {manhattan, euclidean}
Árbol de decisión	`max_depth` $\in$ {1, 3, 5, 10, 12, 13, 14, 20, 25}, `min_samples_leaf` $\in$ {1, 15, 30, 50, 150, 300, 500, 1000}, `ccp_alpha` $\in$ {0.01, 0.005, 0.001, 0.0005, 0.0001}
Random Forest	`n_estimators` $\in$ {10, 50, 100, 500, 1000}, `max_depth` $\in$ {None} $\cup$ {5, 10, 20}, `min_samples_leaf` $\in$ {1, 3, 5, 7, 10},`max_features` $\in$ {None} $\cup$ {1, sqrt, log2}
Regresión Logística	`C` $\in$ {0.0001, 0.0005, 0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1, 5, 10}, `l1_ratio` $\in$ {0.01, 0.05, 0.1, 0.5, 0.9, 0.95, 0.99}
Red neuronal	`hidden_layer_sizes` $\in$ {(10), (25), (100), (25, 10), (10, 10, 10), (25, 25), (50, 10)}, `activation` $\in$ {relu, logistic, tanh}, `learning_rate_init` $\in$ {0.00001, 0.0001, 0.0005, 0.001, 0.005, 0.01, 0.05, 0.1}

Tabla 4.1: Rangos de hiperparámetros considerados en cada modelo.

4.1.6. Métricas

La métrica empleada para la selección de los hiperparámetros ha sido el $F1$. Para la evaluación en test, en cambio, las métricas empleadas han sido algunas de las presentadas en la sección 2.2.9: exactitud, precisión, exhaustividad, $F1$, y AUC:

4.2. Resultados

En esta sección se mostrarán los resultados obtenidos de aplicar las técnicas detalladas en el capítulo 3.

4.2.1. Hiperparámetros seleccionados

En la tabla 4.2 se muestran los valores finales de los hiperparámetros a los que se ha llegado, así como el valor de la métrica $F1$ en validación cruzada.

Modelo	**F1 en CV**	**Mejores hiperparámetros**
KNN	0,909	`n_neighbors` = 101 , `weights` = *distance* , `metric` = *manhattan*
Árbol de decisión	0,907	`max_depth` = 1, `min_samples_leaf` = 50, `ccp_alpha` = 0,0001
Random Forest	0,926	`n_estimators` = 500, `max_depth` = *None*, `min_samples_leaf` = 1, `max_features` = *sqrt*
Regresión Logística	0,859	`C` = 1, `l1_ratio` = 0,5
Red neuronal	0,900	`hidden_layer_sizes` = (50, 10), `activation` = *logistic*, `learning_rate_init` = 0,005

Tabla 4.2: F1-score en validación cruzada y mejores hiperparámetros seleccionados.

4.2.2. Mejores resultados

Los resultados obtenidos sobre el conjunto de prueba se reflejan en la tabla 4.3. Cabe destacar que finalmente se ha optado por estandarizar, puesto que las diferencias en el rendimiento apenas han sido apreciables entre normalizar y estandarizar.

Modelo	**Exactitud**	**Precisión**	**Exhaustividad**	**F1**	**ROC-AUC**
KNN	0,881	0,895	0,932	0,913	0,854
Árbol de decisión	0,887	0,899	0,937	0,917	0,860
Random Forest	0,907	0,915	0,949	0,932	0,884
Regresión Logística	0,812	0,835	0,898	0,865	0,766
Red neuronal	0,878	0,899	0,922	0,910	0,855

Tabla 4.3: Métricas de evaluación en test para cada modelo ajustado.

4.2.3. Discusión

A continuación, tras ver los resultados obtenidos, se analizarán los mismos para tratar de entenderlos con más detalle.

Si se tiene en consideración las métricas $F1score$ y el AUC, entonces la técnica que mejor ha podido predecir las cancelaciones de las reservas ha sido $RandomForest$. Después, el $rboldedecisin$, el KNN y el perceptrón multicapa han tenido un resultado ligeramente inferior al primero. Finalmente, se encuentra la regresión logística con el peor rendimiento de todos.

Es entendible que el *Random Forest* mejore el rendimiento del árbol de decisión, pues no deja de ser una mejora del segundo. También cabe destacar que todas las técnicas han dado lugar a un modelo con un rendimiento aceptable.

Por otro lado, como se ha comentado en la sección 4.2.2, en este caso apenas se pueden apreciar diferencias entre estandarizar los datos y normalizarlos.

En cuanto al posible sobreajuste que pueden estar haciendo los modelos, se puede comparar el valor de la métrica $F1$ en entrenamiento y en prueba. En este caso, dichos valores han 0,995 y 0,932 respectivamente, por lo que sí se puede apreciar un cierto sobreajuste.

Como se puede apreciar en la matriz de confusión presentada en la figura 4.1 del mejor modelo, el accuracy del modelo es de 0,906, por lo que en el 90,6 % de los casos el modelo acierta si la reserva se va a cancelar o no. Por otro lado, se puede observar que para la clase 0 (reserva cancelada) la precisión es de 0,886 y de 0,915 para la clase 1 (reserva no cancelada). Esto quiere decir que, de todas las reservas que el modelo predice que se van a cancelar, acierta en un 88,6 % de los casos, y que cuando predice que no se va a cancelar, acierta en un 91,5 % de las veces.

De la misma forma, la exhaustividad (recall) del modelo para la clase 1 es de 0,949 y de 0,820 para la clase 0, lo cual refleja que el modelo detecta correctamente el 95 % de las reservas que realmente no se cancelan, mientras que identifica el 82 % de las que sí se cancelan. Por tanto, el modelo es más eficaz prediciendo reservas no canceladas que canceladas, aunque en ambos casos presenta un rendimiento aceptable. Esto último puede deberse al hecho de que las clases presentan un desbalance, y por tanto el modelo le da más importancia a la clase 1.

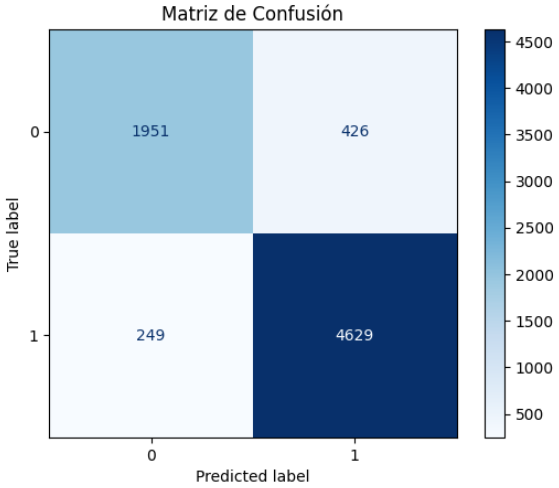

Figura 4.1: Matriz de confusión.

4.3. Estudio de *Overbooking*

A continuación, se ilustra una forma de implementar el modelo obtenido en la sección anterior en la determinación de una política de *overbooking* óptima.

4.3.1. Planteamiento del problema

La tasa de *overbooking* se define como la proporción de reservas que ofrece un establecimiento de alojamiento por encima de la capacidad total:

$$t = \frac{TR - C}{C}$$

donde:

- t = Tasa de *overbooking*
- TR = Número total de reservas vendidas (incluyendo overbooking)
- C = Capacidad real (número de habitaciones disponibles)

Si se dejan habitaciones libres, el establecimiento deja de ganar todo el dinero que podría obtener de llenar todas las habitaciones. Mientras que si el establecimiento decide ofrecer más habitaciones de las que tiene, puede correr el riesgo de que no se cancelen las suficientes reservas y tenga que compensar a los clientes. El beneficio de cada reserva no cancelada se denotará por B y la pérdida por la compensación debido al *overbooking* por P. Por otro lado, la capacidad del establecimiento se denotará por C.

El estudio consistirá en analizar diferentes estrategias para definir una política dinámica de gestión de overbooking para maximizar el beneficio esperado. Para ello, en una de las estrategias se aplica el mejor clasificador de la sección anterior.

4.3.2. Implementación de la solución

A continuación se realizará un estudio de simulación para evaluar diferentes estrategias y ver cuál maximiza los beneficios.

En lugar de fijar de antemano una tasa de *overbooking*, el enfoque considerado en este estudio consiste en ir aceptando reservas una a una hasta que la estrategia aplicada estime que el establecimiento ya se va a llenar.

Cada estrategia proporciona, para cada nueva reserva potencial, una predicción de si esa reserva será efectiva o se cancelará. Utilizando esas predicciones, se va acumulando el número esperado de clientes que finalmente se presentarán (es decir, que no cancelarán). Se dejan de aceptar nuevas reservas cuando se considera que ese número acumulado cubrirá la capacidad total del establecimiento, es decir, cuando se espera que se vayan a ocupar todas las habitaciones. Más adelante se detallará otro enfoque que se adoptará además de este, y que consistirá en fijar un riesgo máximo que se quiere asumir para estimar cúando se va a llenar la capacidad total.

Una vez tomadas todas las decisiones de aceptación de reservas, el resultado observado será un número real de clientes que finalmente se presentan, que se denotará como *contador_real*. Este valor puede ser mayor o menor que la capacidad del establecimiento C, dependiendo de si las reservas canceladas fueron bien estimadas o no. El beneficio final se calcula como:

$$\text{beneficio} = \text{mín}(C, \texttt{contador_real}) \cdot B - \text{máx}(0, \texttt{contador_real} - C) \cdot P$$

De esta forma, se gana B euros por cada cliente que se presenta, hasta un máximo de C habitaciones disponibles. Si el número de clientes que se presentan supera la capacidad C, el exceso debe ser compensado con un coste P por cliente. Por tanto:

- $\text{mín}(C, \texttt{contador_real})$ es el número de clientes que efectivamente pueden alojarse.
- $\text{máx}(0, \texttt{contador_real} - C)$ es el número de clientes que hay que compensar.

Estrategias

El objetivo, por tanto, reside en diseñar la estrategia de decisión que maximice este beneficio esperado al aceptar dinámicamente las reservas. Para ello, se han generado 100 secuencias aleatorias de reservas sobre el conjunto de prueba y, para cada una de ellas, se ha calculado el beneficio obtenido mediante tres estrategias diferentes:

- **Estrategia conservadora** (sin overbooking):

  Esta estrategia evita completamente el *overbooking* y funciona del siguiente modo:

  - Se recorren las reservas secuencialmente.
  - Se lleva un contador del número de reservas aceptadas.
  - Cada vez que llega una nueva reserva, se acepta sólo si el número total de reservas aceptadas es menor que C.
  - Cuando se alcanza la capacidad real del establecimiento, se deja de aceptar reservas.

  Este enfoque garantiza que no haya más reservas que habitaciones disponibles, aunque puede llevar a dejar habitaciones vacías si hay cancelaciones posteriores. Por tanto, se trata de una estrategia consevadora, que en la mayoría de los casos llevará a dejar habitaciones vacías, pero sin correr el riesgo de tener que indemnizar a un huesped que se haya quedado sin una habitación disponible.

- **Estrategia de overbooking fijo** (hasta un tercio por encima de la capacidad):

  En esta estrategia se permite aceptar reservas hasta un número máximo que corresponde a un tercio más de la capacidad, es decir, hasta:

  $$TR_{\text{máx}} = C + \left\lfloor \frac{C}{3} \right\rfloor$$

  El procedimiento es el siguiente:

  - Se recorren las reservas una a una.
  - Se acepta cada reserva mientras el número total de reservas aceptadas sea menor que $TR_{\text{máx}}$.
  - Una vez alcanzado ese límite, se dejan de aceptar reservas.

  Esta estrategia introduce una política de overbooking estática, que asume que aproximadamente un tercio de las reservas podría cancelarse, sin utilizar predicciones individuales. Esto nace de que aproximadamente un tercio de las reservas del conjunto de datos se terminan cancelando, por lo que se puede suponer que se trata de una estrategia más ajustada que la anterior.

- **Estrategia basada en la predicción del mejor modelo obtenido**

  Esta estrategia utiliza el modelo predictivo previamente entrenado (el que mejor rendimiento ha obtenido), que estima la probabilidad de que una reserva no se cancele. El procedimiento consiste en lo siguiente:

  - Se recorren las reservas de forma secuencial.
  - Para cada reserva, se consulta el modelo para obtener la probabilidad de que dicha reserva no se cancele.
  - Se mantiene una variable que acumula la suma de estas probabilidades.
  - Se acepta una reserva si la suma acumulada de las probabilidades no supera la capacidad C.
  - En cuanto la suma acumulada alcanza o supera C, se deja de aceptar nuevas reservas.

  Esta estrategia permite cierto overbooking de forma controlada, confiando en que algunas reservas se cancelarán, según lo predicho por el modelo. Cuanto mejor sea el modelo, más se podrá ajustar la tasa de *overbooking*, y por tanto, el beneficio obtenido será mayor.

Escenarios

Además, con el objetivo de cubrir distintos escenarios plausibles en el contexto real, se ha fijado el beneficio por reserva no cancelada B y se ha variado la penalización por overbooking P, de forma que se exploran diferentes valores de la razón $\frac{B}{P}$. En concreto, se analizan situaciones donde $\frac{B}{P} \leq 1$ (ya que, además de la propia penalización por compensación, se debe tener en cuenta la pérdida de satisfacción del cliente y de imagen de la marca), comenzando por el caso en el que $B = P$, y disminuyendo progresivamente la razón hasta alcanzar valores que representen penalizaciones significativamente mayores que el beneficio:

$$\frac{B}{P} = 1 - 0{,}05 \cdot k, \quad \text{con } k \in \{0, 1, 2, \ldots, 18\}$$

Este análisis permite evaluar la sensibilidad del modelo ante distintos niveles de riesgo económico asociado al overbooking, manteniendo constante el beneficio unitario y modulando únicamente el coste de compensación.

Formulación probabilística de las estrategias

Sea n el número total de reservas candidatas a ser aceptadas. Para cada reserva $i \in \{1, \ldots, n\}$, se define una variable aleatoria:

$$X_i = \begin{cases} 1 & \text{si la reserva } i \text{ no se cancela,} \\ 0 & \text{si la reserva } i \text{ se cancela.} \end{cases}$$

Cada X_i sigue una distribución de Bernoulli con parámetro $p_i = \Pr(X_i = 1)$, que representa la probabilidad de que la reserva no se cancele. Cada estrategia presentada estimará el valor de p_i de una forma distinta:

- *Estrategia conservadora:* Se asume $p_i = 1$ para todo i.
- *Estrategia de overbooking constante:* Se asume $p_i = p$ constante para todo i, con un valor fijo $p = \frac{2}{3}$.
- *Estrategia dinámica:* Se estima p_i utilizando el mejor modelo de clasificación previamente entrenado.

A partir de las variables X_i, se define otra variable aleatoria que representa el número total de reservas no canceladas entre las n aceptadas:

$$Y_n = \sum_{i=1}^{n} X_i$$

Entonces, se pueden construir dos enfoques probabilísticos principales para decidir el número n de reservas a aceptar:

1. **Enfoque basado en la esperanza:**

   Se acepta una nueva reserva mientras la esperanza del número de reservas no canceladas no supere la capacidad del establecimiento:

   $$E[Y_n] = \sum_{i=1}^{n} p_i < C$$

   Cuando $E[Y_n] \geq C$, se deja de aceptar reservas. Esta estrategia busca que, en promedio, no se exceda la capacidad.

2. **Enfoque basado en riesgo controlado:**

   Se define un umbral de riesgo $\alpha \in (0, 1)$, y se acepta una reserva mientras la probabilidad de incurrir en overbooking esté por debajo de ese umbral:

   $$\Pr(Y_n > C) \leq \alpha$$

   Cuando $\Pr(Y_n > C) > \alpha$, se detiene el proceso de aceptación de reservas. Esta estrategia permite controlar el riesgo de sobrepasar la capacidad con una cota probabilística clara, y puede adaptarse a diferentes perfiles de riesgo mediante el ajuste de α. En concreto, para cada valor del ratio entre B y P se ha buscado el valor de α que maximiza el beneficio asociado a la estrategia basada en el modelo, donde:

   $$\alpha \in \{0, 0{,}05, 0{,}1 \ldots, 1\}$$

   Este enfoque resulta especialmente útil cuando el coste asociado a las reservas no canceladas que se hacen por encima de la capacidad total es relativamente mayor al beneficio obtenido por cada reserva no cancelada. Es decir, en estas situaciones, es probable que ser más conservador ayude a no incurrir en *overbooking*, asumiendo el coste de que algunas habitaciones no lleguen a llenarse.

Como punto de referencia, se puede definir una estrategia ideal en la que se conoce con certeza el valor de cada X_i, es decir, si una reserva se cancelará o no. En este caso, se ordenan las reservas y se acepta cada una únicamente si $X_i = 1$, hasta alcanzar la capacidad C. El beneficio obtenido bajo esta estrategia representa el valor máximo alcanzable, y permite evaluar la eficiencia relativa de las estrategias anteriores en escenarios reales.

Finalmente, se ha calculado la media de beneficios y de la tasa de *overbooking* de cada estrategia en cada escenario.

4.3.3. Resultados

Como se puede apreciar en la figura 4.2, la estrategia basada en el predictor ha resultado ser la que más beneficios ha generado y cabe destacar que se ha acercado en más de un 90 % al beneficio máximo teórico en todas las situaciones. Además, se puede apreciar que la tasa de *overbooking* óptima para el conjunto de datos considerado se encuentra cerca del 0,5. La estrategia conservadora ha sido claramente la que peor rendimiento ha tenido, y la estrategia basada en el predictor ha tenido un desempeño ligeramente superior a la estrategia de *overbooking* constante.

Por otro lado, el enfoque basado en la esperanza del número de reservas efectivas se ve penalizado cuando el cociente entre beneficio y penalización, $\frac{B}{P}$, toma valores cercanos a 0, y mejora progresivamente a medida que dicho ratio se aproxima a 1. En cambio, el enfoque dinámico, basado en probabilidades individuales y controlado por un umbral de riesgo α, muestra un rendimiento relativamente mejor incluso con valores bajos de $\frac{B}{P}$, aunque su mejora es menos pronunciada a medida que este ratio aumenta.

Esta diferencia puede explicarse por el hecho de que, para cada valor de $\frac{B}{P}$, se ha optimizado el valor de α, y en general se ha observado que cuando $\frac{B}{P}$ es bajo, el valor óptimo de α también tiende a ser bajo, lo cual se puede apreciar en la figura 4.3. Esto refleja una actitud conservadora frente al riesgo: cuando el beneficio por reserva efectiva es pequeño en comparación con la penalización por exceso de ocupación, conviene aceptar menos reservas para evitar pérdidas. En otras palabras, un valor bajo de α implica un umbral más estricto para el número permitido de reservas, lo que reduce la probabilidad de incurrir en penalizaciones elevadas.

Por el contrario, cuando $\frac{B}{P}$ se aproxima a 1, es decir, cuando el beneficio por cada habitación ocupada se acerca al coste de una penalización por exceso, el sistema tolera mayores niveles de riesgo, y el valor óptimo de α crece hasta aproximadamente 0,25.

Para poder apreciar el efecto que ha tenido la aplicación del modelo de predicción de cancelaciones, en la tabla 4.4 se muestra el beneficio obtenido por cada estrategia para diferentes valores del ratio B/P. Se puede observar que cuando el ratio toma un valor de 0,1, la diferencia entre incorporar el modelo o no, es de 500 unidades monetarias aproximadamente. No obstante, cuanto más grande se hace el valor del ratio, menor es la diferencia entre usar o no el modelo. En el caso de que $B = P$, dicha diferencia pasa a ser de aproximadamente 350 unidades monetarias. Sin embargo, sigue habiendo una diferencia porcentual apreciable de al menos un 3 % en el peor de los casos considerados, lo cual podría implicar que un establecimiento que inicialmente operaba con pérdidas pasara a obtener un saldo neto positivo. Además, en el caso de establecimientos con una capacidad significativamente superior a la considerada en el análisis, si se mantiene esta diferencia relativa, la diferencia absoluta entre aplicar el modelo o no hacerlo resulta aún más destacable.

Estrategia	**Beneficio**	**Tasa**
B/P = 0.10 (Alpha óptimo = 0.05)		
Conservadora	6650.00	0.000
Constante (Esperanza)	7358.00	0.490
Constante (Riesgo)	8956.00	0.360
Dinámica (Esperanza)	7102.00	0.530
Dinámica (Riesgo)	9441.00	0.435
Conocimiento Perfecto	10000.00	0.496
B/P = 0.50 (Alpha óptimo = 0.15)		
Conservadora	6650.00	0.000
Constante (Esperanza)	9246.00	0.490
Constante (Riesgo)	9276.00	0.410
Dinámica (Esperanza)	9358.00	0.530
Dinámica (Riesgo)	9614.00	0.469
Conocimiento Perfecto	10000.00	0.496
B/P = 1.00 (Alpha óptimo = 0.25)		
Conservadora	6650.00	0.000
Constante (Esperanza)	9482.00	0.490
Constante (Riesgo)	9424.00	0.440
Dinámica (Esperanza)	9640.00	0.530
Dinámica (Riesgo)	9716.00	0.491
Conocimiento Perfecto	10000.00	0.496

Tabla 4.4: Resultados para valores distintos de B/P (con alpha óptimo).

Finalmente, también se observa que la tasa de *overbooking* óptima tiende a situarse alrededor de 0,5. Esta observación es coherente con la estrategia de *overbooking* constante: si se espera que una de cada 3 reservas se cancele (es decir, una tasa de cancelación del 33 %), entonces, para una capacidad total de 100 habitaciones, sería razonable aceptar unas 150 reservas. En este caso, la tasa de *overbooking* sería:

$$t = \frac{150 - 100}{100} = 0{,}5.$$

Este valor equilibra el riesgo de cancelaciones con la posibilidad de maximizar la ocupación efectiva del establecimiento.

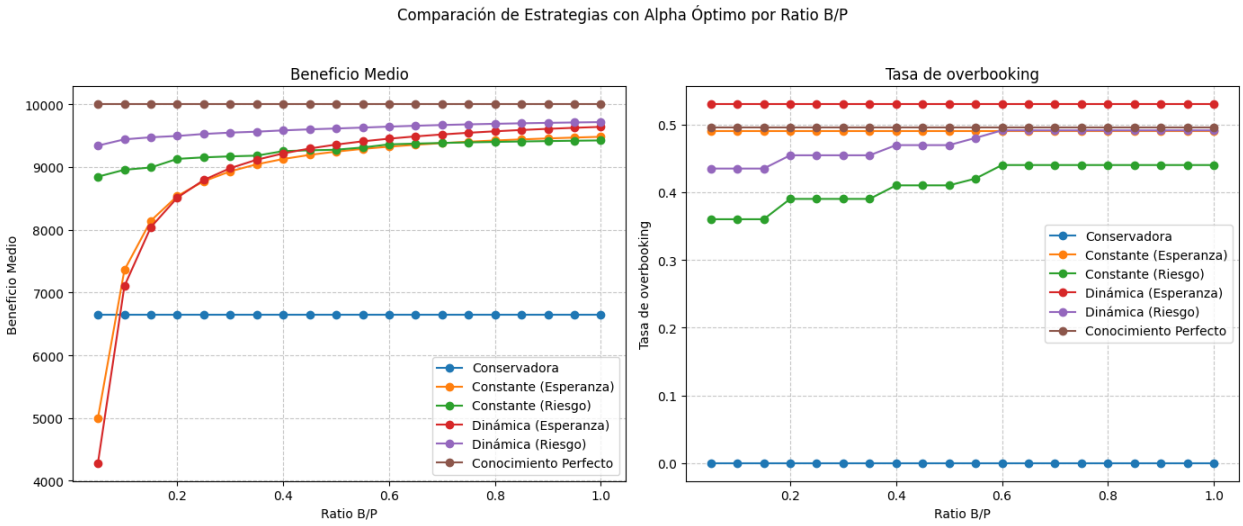

Figura 4.2: Beneficio medio y tasa de overbooking para cada valor de B/P.

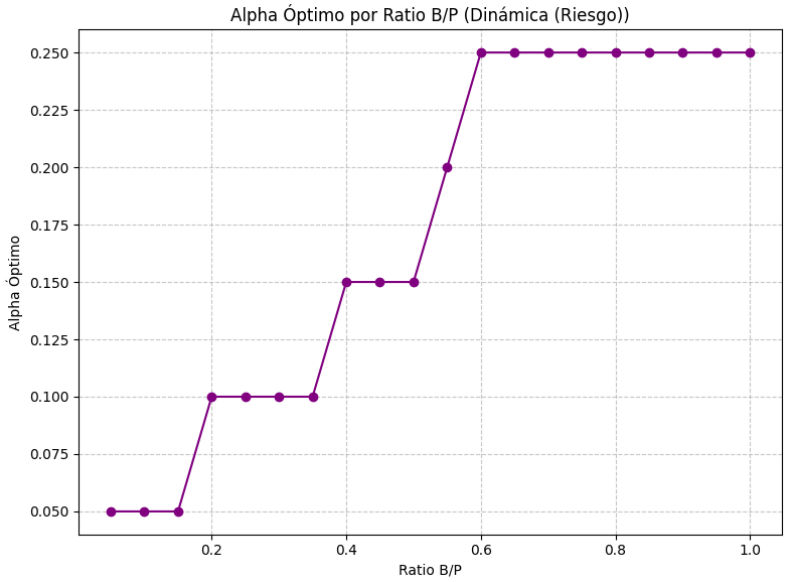

Figura 4.3: Relación entre el ratio B/P y α.

4.3.4. Discusión

Tras ver los resultados obtenidos en el estudio de simulación de diferentes estrategias, se ha podido apreciar que la estrategia desarrollada haciendo uso del modelo ha resultado mejor que el resto bajo una amalgama de escenarios. Por tanto, se puede afirmar que el modelo puede tener un impacto positivo en la maximización de beneficios de las empresas de alojamiento.

Además, el enfoque regido por el riesgo controlado ha resultado ser mejor que el basado en la esperanza, ya que en caso de existir una disparidad entre el beneficio por una reserva y el coste de una reserva por encima de la capacidad, resulta más conveniente ser más conservador y tratar de no incurrir en *overbooking* pese a no llenar todas las reservas.

Cabe destacar que aunque el enfoque de *overbooking* constante, que asume una tasa de overbooking de un 0,5, se acerca al rendimiento de la estrategia regida por el modelo, éste último consigue obtener un rendimiento aún mayor, lo cual es indicativo de que el modelo creado en las secciones anteriores puede tener un impacto positivo en un problema real.

4.4. Mejoras futuras

De cara a futuros trabajos, estas son las consideraciones que se podrían tener en cuenta: por un lado, se podría intentar ajustar otros hiperparámetros que ofrecen las librerías mencionadas. Por otro lado, se podría tratar de partir de un conjunto de datos con mayor volumen de datos, lo cual podría ser beneficioso para algún modelo. Además, se podría tratar de emplear otro tipo de técnicas y ver el rendimiento que éstas ofrecen. Por ejemplo, viendo que los árboles de decisión obtienen un buen rendimiento, técnicas como el *XGBoost* podrían resultar interesantes a primera vista.

Además, cabe destacar la potencial mejora que podrían tener los modelos en caso de que se tuvieran en cuenta variables relacionadas con las condiciones meteorológicas o con los eventos de la zona.

Por otro lado, en cuanto al estudio de la tasa de *overbooking*, se podría tratar de obtener una estrategia teniendo en cuenta otros factores como la demanda esperada o la estacionalidad, con el objetivo de ajustar más los beneficios potenciales. Asimismo, el estudio ganaría robustez en caso de poder tener acceso a datos reales de un hotel y poder ver el impacto real que tendría la aplicación del modelo generado.

Capítulo 5

Conclusión

Tras la realización del trabajo se pueden extraer diversas conclusiones: en primer lugar, se puede observar que las técnicas de aprendizaje supervisado permiten capturar exitosamente relaciones complejas entre grandes volúmenes de datos, como se ha visto en la obtención de un modelo de predicción de cancelaciones de reservas con una tasa de acierto elevada. Por lo tanto, las técnicas de aprendizaje automático con la cantidad creciente de datos que se recopilan actualmente crean una sinergia que permite crear soluciones cada vez más precisas y eficientes.

Por otro lado, se puede apreciar que las técnicas de aprendizaje supervisado se pueden emplear en la optimización de tareas como el ajuste de la tasa de *overbooking*, lo cual tiene un impacto positivo y directo en el sector hotelero. Es decir, estas técnicas, lejos de quedar en un plano teórico, tienen una aplicación directa en el mundo real y están más vigentes que nunca.

No obstante, también se han identificado algunas limitaciones. Por ejemplo, el rendimiento del modelo depende en gran medida de la calidad y representatividad del conjunto de datos. Además, el enfoque seguido no tiene en cuenta la incertidumbre económica asociada a las cancelaciones ni el impacto potencial en la satisfacción del cliente, aspectos que podrían explorarse en futuras ampliaciones del trabajo.

Como líneas futuras, podría ser interesante añadir variables externas, como condiciones meteorológicas o eventos locales, que potencialmente podrían influir en las decisiones de los clientes. También se podrían evaluar otras técnicas más avanzadas, para mejorar la precisión del sistema. Finalmente, la integración del modelo en un sistema de reservas real permitiría validar su utilidad en un entorno práctico y en tiempo real.

En definitiva, este trabajo pone de manifiesto el potencial del aprendizaje automático como herramienta de apoyo a la toma de decisiones en entornos complejos y competitivos como el sector hotelero, y abre la puerta a desarrollos futuros que amplíen su alcance y aplicabilidad.

Bibliografía

[1] Shaza M Abd Elrahman and Ajith Abraham. A review of class imbalance problem. *Journal of Network and Innovative Computing*, 1:9–9, 2013.

[2] Nuno Antonio, Ana de Almeida, and Luis Nunes. An automated machine learning based decision support system to predict hotel booking cancellations. *Data Science Journal*, 18:32–32, 2019.

[3] Gustavo E. A. P. A. Batista, Ronaldo C. Prati, and Maria Carolina Monard. A study of the behavior of several methods for balancing machine learning training data. *SIGKDD Explor. Newsl.*, 6(1):20–29, June 2004.

[4] Aditya Bhardwaj, Tanishq Yadav, and Rajat Chaudhary. Predicting hotel booking cancellations using machine learning techniques. In *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pages 1–6. IEEE, 2024.

[5] Tom Dietterich. Overfitting and undercomputing in machine learning. *ACM Computing Surveys*, 27:1,2, 1995.

[6] Alien Embarec Riadi et al. Aprendizaje automático aplicado al sector hotelero. 2020.

[7] Damien Ernst and Arthur Louette. Introduction to reinforcement learning. *Feuerriegel, S., Hartmann, J., Janiesch, C., and Zschech, P*, pages 111–126, 2024.

[8] Osisanwo F.Y., Akinsola J.E.T., Awodele O., Hinmikaiye J. O., Olakanmi O., and Akinjobi J. Supervised machine learning algorithms: Classification and comparison. *International Journal of Computer Trends and Technology (IJCTT)*, 48:2, 2017.

[9] Miroslav Kubat. *An introduction to Machine Learning*, volume 2nd Ed. Springer, 2017.

[10] Farhad Maleki, Nikesh Muthukrishnan, Katie Ovens, Caroline Reinhold, and Reza Forghani. Machine learning algorithm validation from essentials to advanced applications and implications for regulatory certification and deployment. *Neuroimaging Clinics*, 30:433–445, 2020.

[11] Ahsan Raza. Hotels reservations dataset, 2024.

[12] Ludvin Eduardo Rueda Rincón. Estudio comparativo entre diferentes algoritmos de clasificación para realizar la predicción de las cancelaciones de reservas en un hotel en ciénaga magdalena. B.S. thesis, ESCUELA DE INGENIERÍAS Y ARQUITECTURA, 2022.

[13] Agustín J. Sánchez-Medina and Eleazar C-Sánchez. Using machine learning and big data for efficient forecasting of hotel booking cancellations. *International Journal of Hospitality Management*, 89:102546, 2020.

[14] Yingjie Tian and Yuqi Zhang. A comprehensive survey on regularization strategies in machine learning. *Information Fusion*, 80:146–166, 2022.

[15] Robert Tibshirani Trevor Hastie and Jerome Friedman. *The Elements of Statistical Learning*, volume 2nd Ed. Springer, 2009.

[16] Ž. Vujović. Classification model evaluation metrics. *International Journal of Advanced Computer Science and Applications*, 12, 2021.

[17] Li Yang and Abdallah Shami. On hyperparameter optimization of machine learning algorithms: Theory and practice. *Neurocomputing*, 415:295–316, 2020.

Apéndice A

Outliers

Para detectar valores atípicos en una variable numérica, se ha empleado el método del rango intercuartílico (IQR). Este procedimiento consiste en calcular los cuartiles e identificar como outliers aquellos valores que se encuentran significativamente alejados de la mayoría de los datos. Dado un conjunto de datos, se definen:

- Q_1: primer cuartil (percentil 25)
- Q_3: tercer cuartil (percentil 75)
- $\text{IQR} = Q_3 - Q_1$: rango intercuartílico

Entonces, los outliers son aquellos valores x que cumplen:

$$x < Q_1 - 1{,}5 \cdot \text{IQR} \quad \text{o} \quad x > Q_3 + 1{,}5 \cdot \text{IQR}$$

Apéndice B

Código

A continuación se mostrará el código que se ha empleado para llevar a cabo el estudio práctico. En primer lugar, se mostrará el código referente al preprocesamiento de los datos.

```
[ ]  import pandas as pd
     import numpy as np
     from google.colab import drive
     drive.mount('/content/drive')

Mounted at /content/drive
```

Análisis de los datos

En primer lugar cargamos el conjunto de datos

```
df = pd.read_csv('/content/drive/My Drive/TFG MATES/Hotel_Reservations.csv')
print(df)
```

Mostrar salida oculta

A continuación veamos el número de valores que faltan

```
[ ]  print(df.isnull().sum())
```

Mostrar salida oculta

Vemos que el dataset no contiene valores que faltan. Esto es comprensible puesto que este tipo de datasets suelen tener una alta usabilidad y ya están depurados para su uso.

Veamos si hay datos duplicados

```
[ ]  print(df.duplicated().sum())
```

Mostrar salida oculta

No hay datos duplicados

Veamos si las dos clases están balanceadas

```
[ ]  print(df['booking_status'].value_counts(normalize = True))
```

Mostrar salida oculta

Se puede observar que no están balanceadas, pues hay el doble de reservas no canceladas que canceladas.

Creemos variables derivadas que podrán ayudar a una mejor clasificación

```
[ ]  df['porcentaje_cancelaciones'] =  np.where((df['no_of_previous_bookings_not_canceled'] + df['no_of_previous_cancellations']) != 0, df['no_of_previous_cancellations'] / (df['no_of_previous_bookings_not_canceled']+ df['no_of_previous_cancellations']), 0)
     df.to_csv('/content/drive/My Drive/TFG MATES/nueva_var.csv', index=False)
```

Veamos el número de atributos categóricos y numéricos que tiene el dataset: El dataset contiene 5 atributos categoricos y la variable a predecir, y el resto son numéricos.

Veamos la correlación que existe entre diferentes atributos. Si existe una alta correlación en valor absoluto, entonces incluyendo una variable es suficiente, pues incluir la segunda no ofrece mayor información.

```
import seaborn as sns
import matplotlib.pyplot as plt

correlation_matrix =  df.drop(columns=['Booking_ID', 'type_of_meal_plan', 'room_type_reserved', 'market_segment_type', 'booking_status']).corr()


plt.figure(figsize=(60, 50))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.show()
```

Mostrar salida oculta

Se puede apreciar que obviando algún atributo que tiene una correlación en valor absoluto cercano a 0.5, el resto apenas tiene correlación entre sí.

Veamos la distribución de las variables

```
[ ] print(df.describe())
```

Mostrar salida oculta

Aparentemente todos los datos son coherentes y no hay ninguno fuera de lo normal.

```
[ ] for columna in df.select_dtypes(exclude=['number']).columns:
        # Ver los valores únicos de cada columna no numérica
        print(f"Valores únicos en la columna '{columna}':")
        print(df[columna].unique())
```

Mostrar salida oculta

Podemos ver que hay 4 tipos de planes de comida (desde **Meal Plan 1** hasta **Meal plan 2**), 7 tipos de habitaciones (desde *Room_Type 1*) hasta **Room_Type 7** y 5 tipos de tipos de reservas ('**Offline**, **Online**, **Corporate**, **Aviation** y **Complementary**)

Examinemos si hay outlayers

```
[ ] for columna in df.select_dtypes(include=['number']).columns:
        Q1 = df[columna].quantile(0.25)
        Q3 = df[columna].quantile(0.75)
        IQR = Q3 - Q1
        outliers = df[(df[columna] < (Q1 - 1.5 * IQR)) | (df[columna] > (Q3 + 1.5 * IQR))]

        print(f"{columna}: {outliers.shape[0]}")
```

Mostrar salida oculta

Se puede apreciar que en general no existen muchos outliers, pero **no_of_adults** tiene en torno a un 30% de valores atípicos. No obstante, como se ha visto anteriormente, todos los datos parecen estar en rangos realistas.

Usemos One-hot encoding para las variables categóricas **type_of_meal_plan**, **room_type_reserved** y **market_segment_ty**

```
[ ] df_one_hot = pd.get_dummies(df, columns=['type_of_meal_plan', 'room_type_reserved', 'market_segment_type'])
    df_one_hot.to_csv('/content/drive/My Drive/TFG MATES/one_hot.csv', index=False)
```

Usemos label encoding para las columnas **arrival_year** y **booking_status**

```
[ ] from sklearn.preprocessing import LabelEncoder
    label_encoder = LabelEncoder()
    df = pd.read_csv('/content/drive/My Drive/TFG MATES/one_hot.csv')
    df['arrival_year'] = label_encoder.fit_transform(df['arrival_year']) # 0:2017 y 1:2018
    df['booking_status'] = label_encoder.fit_transform(df['booking_status']) # 0:Canceled y 1:Not_canceled
    df.to_csv('/content/drive/My Drive/TFG MATES/label_encoder.csv', index=False)
```

Normalicemos y/o estandaricemos los datos

```
[ ] from sklearn.preprocessing import MinMaxScaler
    from sklearn.preprocessing import StandardScaler

    scaler_1 = MinMaxScaler()
    scaler_2 = StandardScaler()
    df = pd.read_csv('/content/drive/My Drive/TFG MATES/label_encoder.csv')
    df[['no_of_adults', 'no_of_children', 'no_of_weekend_nights','no_of_week_nights', 'lead_time', 'arrival_month', 'arrival_date', 'no_of_previous_cand
    df.to_csv('/content/drive/My Drive/TFG MATES/normalizado.csv', index=False)
```

Usemos One-hot encoding para las variables categóricas **type_of_meal_plan**, **room_type_reserved** y **market_segment_type**

```
[ ] df_one_hot = pd.get_dummies(df, columns=['type_of_meal_plan', 'room_type_reserved', 'market_segment_type'])
    df_one_hot.to_csv('/content/drive/My Drive/TFG MATES/one_hot.csv', index=False)
```

Usemos label encoding para las columnas **arrival_year** y **booking_status**

```
[ ] from sklearn.preprocessing import LabelEncoder
    label_encoder = LabelEncoder()
    df = pd.read_csv('/content/drive/My Drive/TFG MATES/one_hot.csv')
    df['arrival_year'] = label_encoder.fit_transform(df['arrival_year']) # 0:2017 y 1:2018
    df['booking_status'] = label_encoder.fit_transform(df['booking_status']) # 0:Canceled y 1:Not_canceled
    df.to_csv('/content/drive/My Drive/TFG MATES/label_encoder.csv', index=False)
```

Normalicemos y/o estandaricemos los datos

```
[ ] from sklearn.preprocessing import MinMaxScaler
    from sklearn.preprocessing import StandardScaler

    scaler_1 = MinMaxScaler()
    scaler_2 = StandardScaler()
    df = pd.read_csv('/content/drive/My Drive/TFG MATES/label_encoder.csv')
    df[['no_of_adults', 'no_of_children', 'no_of_weekend_nights','no_of_week_nights', 'lead_time', 'arrival_month', 'arrival_date', 'no_of_previous_cancellations', 'no_of_previous_bookings_not_canceled', 'avg_price_per_room', 'no_of_special_requests']] \
    = scaler_1.fit_transform(df[['no_of_adults', 'no_of_children', 'no_of_weekend_nights', 'no_of_week_nights', 'lead_time', 'arrival_month', 'arrival_date', 'no_of_previous_cancellations', 'no_of_previous_bookings_not_canceled', 'avg_price_per_room', \
                                 'no_of_special_requests']])
    df.to_csv('/content/drive/My Drive/TFG MATES/normalizado.csv', index=False)

    df = pd.read_csv('/content/drive/My Drive/TFG MATES/label_encoder.csv')
    df[['no_of_adults', 'no_of_children', 'no_of_weekend_nights','no_of_week_nights', 'lead_time', 'arrival_month', 'arrival_date', 'no_of_previous_cancellations', 'no_of_previous_bookings_not_canceled', 'avg_price_per_room', 'no_of_special_requests']] \
    = scaler_2.fit_transform(df[['no_of_adults', 'no_of_children', 'no_of_weekend_nights', 'no_of_week_nights', 'lead_time', 'arrival_month', 'arrival_date', 'no_of_previous_cancellations', 'no_of_previous_bookings_not_canceled', 'avg_price_per_room', \
                                 'no_of_special_requests']])
    df.to_csv('/content/drive/My Drive/TFG MATES/estandarizado.csv', index=False)
```

```
[ ] df = pd.read_csv('/content/drive/My Drive/TFG MATES/normalizado.csv')
    df = df.drop(columns=['porcentaje_cancelaciones'])
    df.to_csv('/content/drive/My Drive/TFG MATES/normalizado_sin_var.csv', index=False)
    df = pd.read_csv('/content/drive/My Drive/TFG MATES/estandarizado.csv')
    df = df.drop(columns=['porcentaje_cancelaciones'])
    df.to_csv('/content/drive/My Drive/TFG MATES/estandarizado_sin_var.csv', index=False)
```

Con esto hemos terminado el preprocesamiento de los datos. Ahora comenzaremos con el entrenamiento de los modelos

A continuación se mostrará el código más relevante referente a la creación de los modelos.

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import GridSearchCV
import time
from itertools import product

def ajustar_modelos(archivo):
    df = pd.read_csv(archivo)
    X = df.drop(columns=['booking_status', 'Booking_ID'])
    y = df['booking_status']
    X_ent, X_pru, y_ent, y_pru = train_test_split(X, y, test_size=0.2, random_state=42, stratify = y)

    modelos = {
        'KNN': {
            'modelo': KNeighborsClassifier(),
            'hiperparametros': {
                'n_neighbors': [1, 3, 5, 11, 21, 31, 51, 101],
                'weights': ['distance', 'uniform'],
                'metric': ['manhattan', 'euclidean']
            }
        } ,
        'Árbol de decisión':{
            'modelo': DecisionTreeClassifier(),
            'hiperparametros':{
                'max_depth': [1, 3, 5, 10, 12, 13,14, 20, 25],
                'min_samples_leaf': [1, 15, 30, 50, 150, 300, 500, 1000],
                'ccp_alpha' : [0.01, 0.005, 0.001, 0.0005, 0.0001]
            }
        },
        'Random Forest': {
            'modelo': RandomForestClassifier(),
            'hiperparametros': {
                'max_depth': [None, 5, 10, 20],
                'min_samples_leaf': [1, 3, 5, 7, 10],
                'n_estimators': [10, 50, 100, 500, 1000],
                'max_features': [1, 'sqrt', 'log2', None],
            }
        },
```

```
    'Regresión Logística':{
        'modelo': LogisticRegression(),
        'hiperparametros': {
            'C': [0.0001, 0.0005, 0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1, 5, 10],
            'penalty': ['elasticnet'],
            'solver': ['saga'],
            'l1_ratio':  [0.01, 0.05, 0.1, 0.5, 0.9, 0.95, 0.99],
        }
    },
    'Red neuronal': {
        'modelo': MLPClassifier(),
        'hiperparametros': {
            'hidden_layer_sizes': [(10), (25), (100), (25, 10), (10, 10, 10), (25, 25), (50, 10)],
            'activation': ['logistic'],
            'learning_rate_init': [0.00001, 0.0001, 0.0005, 0.001, 0.005, 0.01, 0.05, 0.1],
        }
    }
}




hiperparametros_lista = []
for modelo, valores in modelos.items():
    for param, valores_param in valores['hiperparametros'].items():
        hiperparametros_lista.append([modelo, param, valores_param])

df_hiperparametros = pd.DataFrame(hiperparametros_lista, columns=['Modelo', 'Hiperparámetro', 'Valores'])
print("Tabla 1: Hiperparámetros explorados")
print(df_hiperparametros.to_string())




resultados = {
    'Modelo': ['KNN','Árbol de decisión', 'Random Forest', 'Regresión Logística','Red neuronal'],
    'Exactitud': [],
    'Precision': [],
    'Exhaustividad': [],
    'F1': [],
    'ROC AUC': [],
    'Tiempo': [],
    'Mejores hiperparámetros': []
  }
resultados_cv = {
    'Modelo': ['KNN','Árbol de decisión', 'Random Forest', 'Regresión Logística','Red neuronal'],
    'media': [],
    'desviación': []
}
```

```
for nombre, modelo in modelos.items():
    grid_search = GridSearchCV(modelo['modelo'], modelo['hiperparametros'], cv=5, scoring="f1", n_jobs=-1, verbose = 4)
    ini = time.time()
    grid_search.fit(X_ent, y_ent)
    tiempo = time.time() - ini
    y_pred = grid_search.best_estimator_.predict(X_pru)
    accuracy = accuracy_score(y_pru, y_pred)
    precision = precision_score(y_pru, y_pred)
    recall = recall_score(y_pru, y_pred)
    f1 = f1_score(y_pru, y_pred)
    roc_auc = roc_auc_score(y_pru, y_pred)
    resultados['Exactitud'].append(accuracy)
    resultados['Precision'].append(precision)
    resultados['Exhaustividad'].append(recall)
    resultados['F1'].append(f1)
    resultados['ROC AUC'].append(roc_auc)
    resultados['Tiempo'].append(tiempo)
    results_df = pd.DataFrame(grid_search.cv_results_)

    resultados['Mejores hiperparámetros'].append(grid_search.best_params_)
    mejor_indice = grid_search.best_index_
    mejor_resultado = grid_search.cv_results_
    resultados_cv['media'].append(mejor_resultado['mean_test_score'][mejor_indice])
    resultados_cv['desviación'].append(mejor_resultado['std_test_score'][mejor_indice])

df_resultados = pd.DataFrame(resultados)
df_resultados.to_csv('/content/drive/My Drive/TFG MATES/resultados_en_test.csv', index=False)
df_resultados_cv = pd.DataFrame(resultados_cv)
df_resultados_cv.to_csv('/content/drive/My Drive/TFG MATES/resultados_en_test.csv', index=False)

print(df_resultados.to_string())
print(df_resultados_cv.to_string())
```

```python
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

modelo = RandomForestClassifier(random_state=42, min_samples_leaf = 1, max_features = 'sqrt', n_estimators=500)
df = pd.read_csv('/content/drive/My Drive/TFG MATES/estandarizado.csv')
X = df.drop(columns=['booking_status', 'Booking_ID'])
y = df['booking_status']
X_ent, X_pru, y_ent, y_pru = train_test_split(X, y, test_size=0.2, random_state=42, stratify = y)
modelo.fit(X_ent, y_ent)
y_pred = modelo.predict(X_pru)

cm = confusion_matrix(y_pru, y_pred)

print("Matriz de confusión:")
print(cm)

disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues')
plt.title("Matriz de Confusión")
plt.show()
```

```python
modelo = RandomForestClassifier(random_state=42, min_samples_leaf = 1, max_features = 'sqrt', n_estimators=500)
df = pd.read_csv('/content/drive/My Drive/TFG MATES/estandarizado.csv')
X = df.drop(columns=['booking_status', 'Booking_ID'])
y = df['booking_status']
X_ent, X_pru, y_ent, y_pru = train_test_split(X, y, test_size=0.2, random_state=42, stratify = y)
modelo.fit(X_ent, y_ent)
y_train_pred = modelo.predict(X_ent)
f1 = f1_score(y_train_pred, y_ent)
print(f1)
```

```python
modelo = RandomForestClassifier(random_state=42, min_samples_leaf = 1, max_features = 'sqrt', n_estimators=500)

df = pd.read_csv('/content/drive/My Drive/TFG MATES/estandarizado.csv')
X = df.drop(columns=['booking_status', 'Booking_ID'])
y = df['booking_status']
X_ent, X_pru, y_ent, y_pru = train_test_split(X, y, test_size=0.2, random_state=42, stratify = y)
modelo.fit(X_ent, y_ent)
y_pred = modelo.predict(X_pru)
df_caso_practico = pd.DataFrame({
    'y_real': y_pru,
    'y_pred': y_pred
})

print(df_caso_practico)
df_caso_practico.to_csv('/content/drive/My Drive/TFG MATES/caso_practico.csv', index=False)
```

Para finalizar, se mostrará el código utilizado para el problema de la tasa de *overbooking*.

```
import numpy as np
import pandas as pd
from scipy.stats import norm
import matplotlib.pyplot as plt

np.random.seed(42)
capacidad = 100
beneficio_por_reserva = 100
prob_constante = 2/3
ratios_b_p = np.arange(0.05, 1.05, 0.05)
alphas = np.arange(0, 1.05, 0.05)

# Funciones para las estrategias
def calcular_esperanza_y_varianza(probabilidades, n):
    esperanza = np.sum(probabilidades[:n])
    varianza = np.sum(probabilidades[:n] * (1 - probabilidades[:n]))
    return esperanza, varianza

def estrategia_por_esperanza(probabilidades, capacidad):
    esperanza = 0
    n = 0
    while n < len(probabilidades) and esperanza < capacidad:
        esperanza += probabilidades[n]
        n += 1
    return n - 1 if esperanza >= capacidad else n

def estrategia_por_riesgo(probabilidades, capacidad, alpha):
    n = 0
    while n < len(probabilidades):
        esperanza, varianza = calcular_esperanza_y_varianza(probabilidades, n + 1)
        if varianza == 0:
            prob_sobre_reserva = 1.0 if esperanza > capacidad else 0.0
        else:
            z = (capacidad - esperanza) / np.sqrt(varianza)
            prob_sobre_reserva = 1 - norm.cdf(z)
        if prob_sobre_reserva > alpha:
            break
        n += 1
    return n

def estrategia_conocimiento_perfecto(valores_reales, capacidad):
    n = 0
    aceptadas = 0
    while n < len(valores_reales) and aceptadas < capacidad:
        if valores_reales[n] == 1:
            aceptadas += 1
        n += 1
    return n
```

```python
# Función para simular una estrategia y calcular métricas
def simular_estrategia(estrategia, df, capacidad, beneficio_por_reserva, penalizacion, alpha, n_iter=100):
    tasas_sobre_reserva = []
    beneficios = []

    for i in range(n_iter):
        df_mezclado = df.sample(frac=1, random_state=i).reset_index(drop=True)
        prob_predichas = df_mezclado['y_proba'].values
        valores_reales = df_mezclado['y_real'].values
        n = 0
        if estrategia == "Conservadora":
            n = capacidad
        elif estrategia == "Constante (Esperanza)":
            n = estrategia_por_esperanza(np.full(len(df), prob_constante), capacidad)
        elif estrategia == "Constante (Riesgo)":
            n = estrategia_por_riesgo(np.full(len(df), prob_constante), capacidad, alpha)
        elif estrategia == "Dinámica (Esperanza)":
            n = estrategia_por_esperanza(prob_predichas, capacidad)
        elif estrategia == "Dinámica (Riesgo)":
            n = estrategia_por_riesgo(prob_predichas, capacidad, alpha)
        elif estrategia == "Conocimiento Perfecto":
            n = estrategia_conocimiento_perfecto(valores_reales, capacidad)

        reservas_reales = np.sum(valores_reales[:n])
        beneficio = min(capacidad, reservas_reales) * beneficio_por_reserva - max(0, reservas_reales - capacidad) * penalizacion
        tasa = (n - capacidad) / capacidad

        tasas_sobre_reserva.append(tasa)
        beneficios.append(beneficio)

    return {
        'tasa_media': float(np.mean(tasas_sobre_reserva)),
        'beneficio_medio': float(np.mean(beneficios))
    }
```

```python
# Función principal para ejecutar la simulación y seleccionar el mejor alpha
def ejecutar_simulacion(archivo):
    df = pd.read_csv(archivo)

    if 'y_proba' not in df.columns or 'y_real' not in df.columns:
        raise ValueError("El DataFrame debe contener las columnas 'y_proba' y 'y_real'")

    resultados_totales = {}

    for alpha in alphas:
        resultados_alpha = {}
        for ratio in ratios_b_p:
            penalizacion = beneficio_por_reserva / ratio
            resultados_alpha[f"B/P={ratio:.2f}"] = simular_estrategias(df, capacidad, beneficio_por_reserva, penalizacion, alpha)
        resultados_totales[f"alpha={alpha:.2f}"] = resultados_alpha

    mejores_alphas = {}
    for ratio in ratios_b_p:
        clave_ratio = f"B/P={ratio:.2f}"
        mejor_alpha = None
        mejor_beneficio = -np.inf
        mejor_tasa = None

        for alpha in alphas:
            clave_alpha = f"alpha={alpha:.2f}"
            beneficio = resultados_totales[clave_alpha][clave_ratio]["Dinámica (Riesgo)"]['beneficio_medio']
            if beneficio > mejor_beneficio:
                mejor_beneficio = beneficio
                mejor_alpha = alpha
                mejor_tasa = resultados_totales[clave_alpha][clave_ratio]["Dinámica (Riesgo)"]['tasa_media']

        mejores_alphas[clave_ratio] = {
            'alpha': mejor_alpha,
            'beneficio': mejor_beneficio,
            'tasa': mejor_tasa
        }

    print("\nMejor alpha para cada ratio B/P (Estrategia Dinámica (Riesgo)):")
    print("Ratio B/P | Mejor Alpha | Beneficio Medio | Tasa de Overbooking")
    print("-" * 65)
    for clave_ratio, metricas in mejores_alphas.items():
        print(f"{clave_ratio:9} | {metricas['alpha']:.2f}         | {metricas['beneficio']:.2f}          | {metricas['tasa']:.3f}")

    resultados_comparacion = {}
    for ratio in ratios_b_p:
        clave_ratio = f"B/P={ratio:.2f}"
        alpha_optimo = mejores_alphas[clave_ratio]['alpha']
        penalizacion = beneficio_por_reserva / ratio
        resultados_comparacion[clave_ratio] = simular_estrategias(df, capacidad, beneficio_por_reserva, penalizacion, alpha_optimo)
```

```
print("\nComparación de estrategias usando el mejor alpha para cada ratio B/P:")
for clave_ratio, resultados in resultados_comparacion.items():
    print(f"\n{clave_ratio} (Alpha óptimo = {mejores_alphas[clave_ratio]['alpha']:.2f}):")
    for estrategia, metricas in resultados.items():
        print(f"  {estrategia}: Beneficio = {metricas['beneficio_medio']:.2f}, Tasa = {metricas['tasa_media']:.3f}")

estrategias = [
    "Conservadora",
    "Constante (Esperanza)",
    "Constante (Riesgo)",
    "Dinámica (Esperanza)",
    "Dinámica (Riesgo)",
    "Conocimiento Perfecto"
]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
fig.suptitle('Comparación de Estrategias con Alpha Óptimo por Ratio B/P')

for estrategia in estrategias:
    beneficios = [resultados_comparacion[f"B/P={ratio:.2f}"][estrategia]['beneficio_medio'] for ratio in ratios_b_p]
    ax1.plot(ratios_b_p, beneficios, marker='o', label=estrategia)
ax1.set_xlabel('Ratio B/P')
ax1.set_ylabel('Beneficio Medio')
ax1.set_title('Beneficio Medio')
ax1.legend()
ax1.grid(True, linestyle='--', alpha=0.7)

for estrategia in estrategias:
    tasas = [resultados_comparacion[f"B/P={ratio:.2f}"][estrategia]['tasa_media'] for ratio in ratios_b_p]
    ax2.plot(ratios_b_p, tasas, marker='o', label=estrategia)
ax2.set_xlabel('Ratio B/P')
ax2.set_ylabel('Tasa de overbooking')
ax2.set_title('Tasa de overbooking')
ax2.legend()
ax2.grid(True, linestyle='--', alpha=0.7)

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()

fig, ax = plt.subplots(figsize=(8, 6))
alphas_optimos = [mejores_alphas[f"B/P={ratio:.2f}"]['alpha'] for ratio in ratios_b_p]
ax.plot(ratios_b_p, alphas_optimos, marker='o', color='purple')
ax.set_xlabel('Ratio B/P')
ax.set_ylabel('Alpha Óptimo')
ax.set_title('Alpha Óptimo por Ratio B/P (Dinámica (Riesgo))')
ax.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

return mejores_alphas, resultados_comparacion
```